

Compilers' Theory

Lecture 2

Prof. Zaki Taha

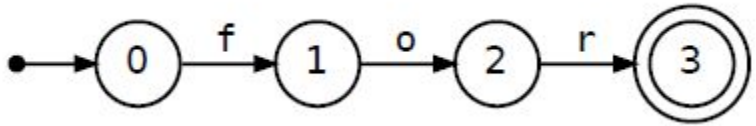
Course Coordinator
Dr. Mohamed Ali
mohamed.abdelghafour@cis.asu.edu.eg



Finite Automata

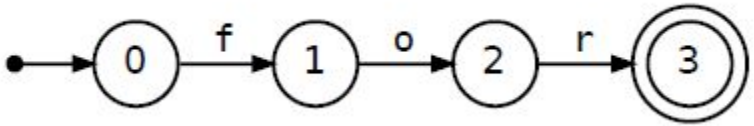
What is a Finite Automaton (FA)?

- An abstract mathematical machine representing state-based computation.
- Graphical Representation:
  - **States**: Numbered circles representing configurations.
  - **Edges**: Directed arrows labeled with symbols representing transitions.
  - **Start State** (S_0): Marked with an incoming arrow.
  - **Accepting States**: Indicated by a double circle.
- There is a strong relationship between finite automata and regular expressions.



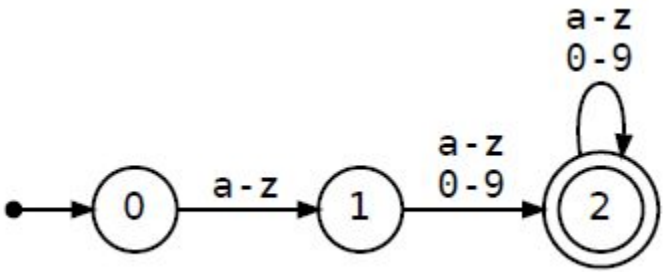
Finite Automaton (FA)

- An FA for the keyword for



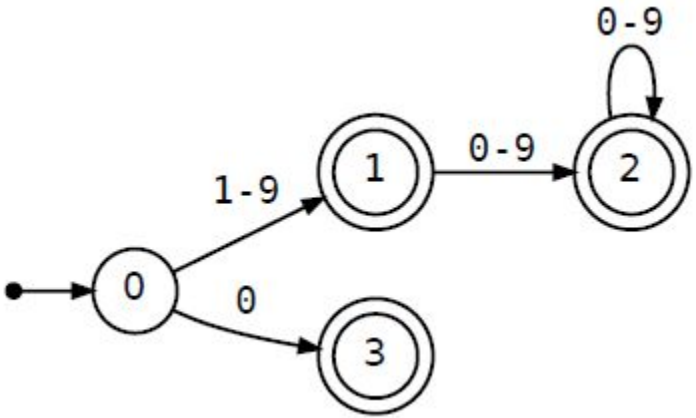
Finite Automaton (FA)

- An FA for identifiers of the form $[a\text{-}z][a\text{-}z0\text{-}9]^+$



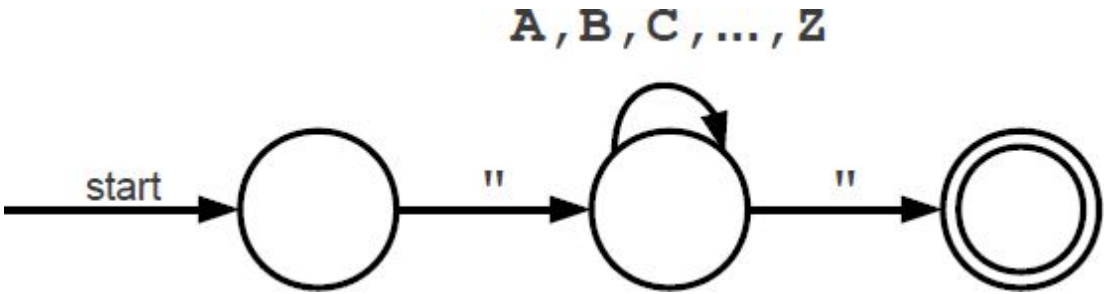
Finite Automaton (FA)

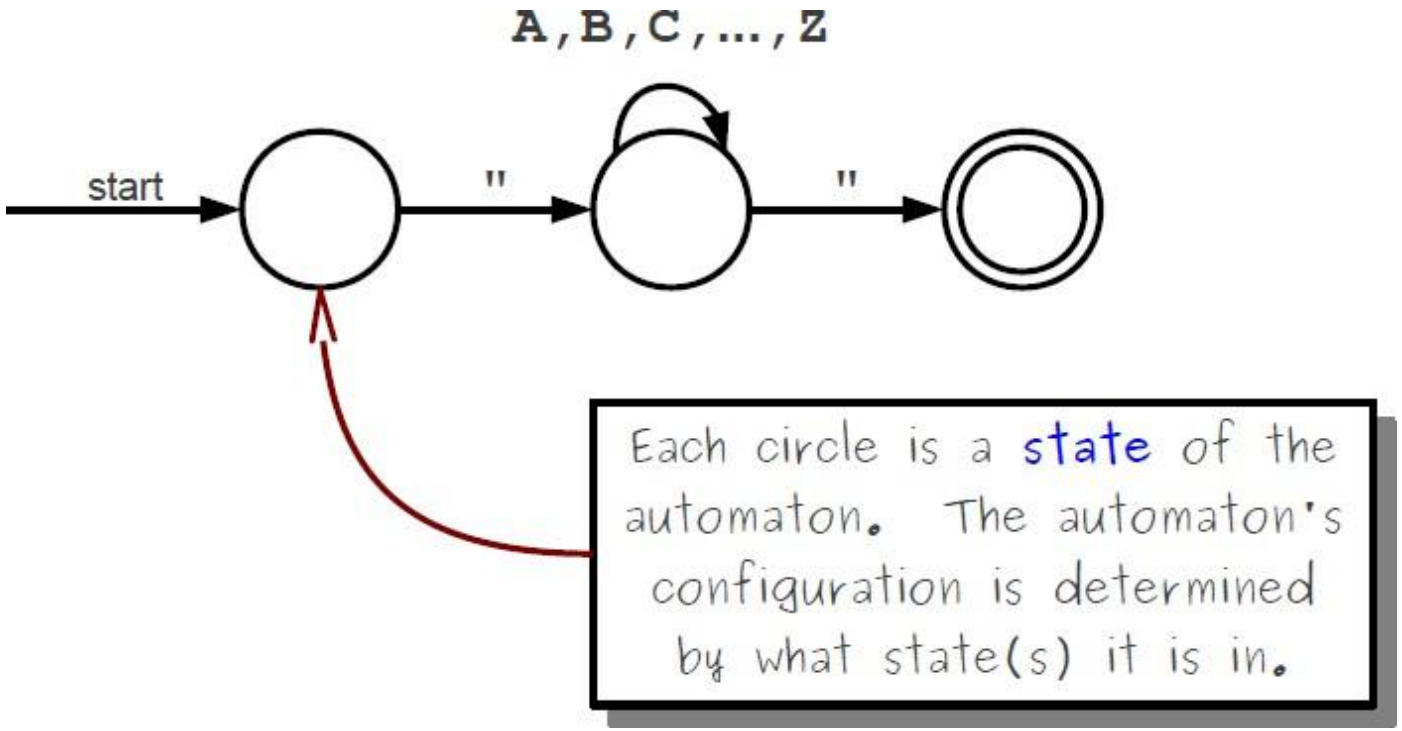
- An FA for numbers of the form ([1-9][0-9]*)|0

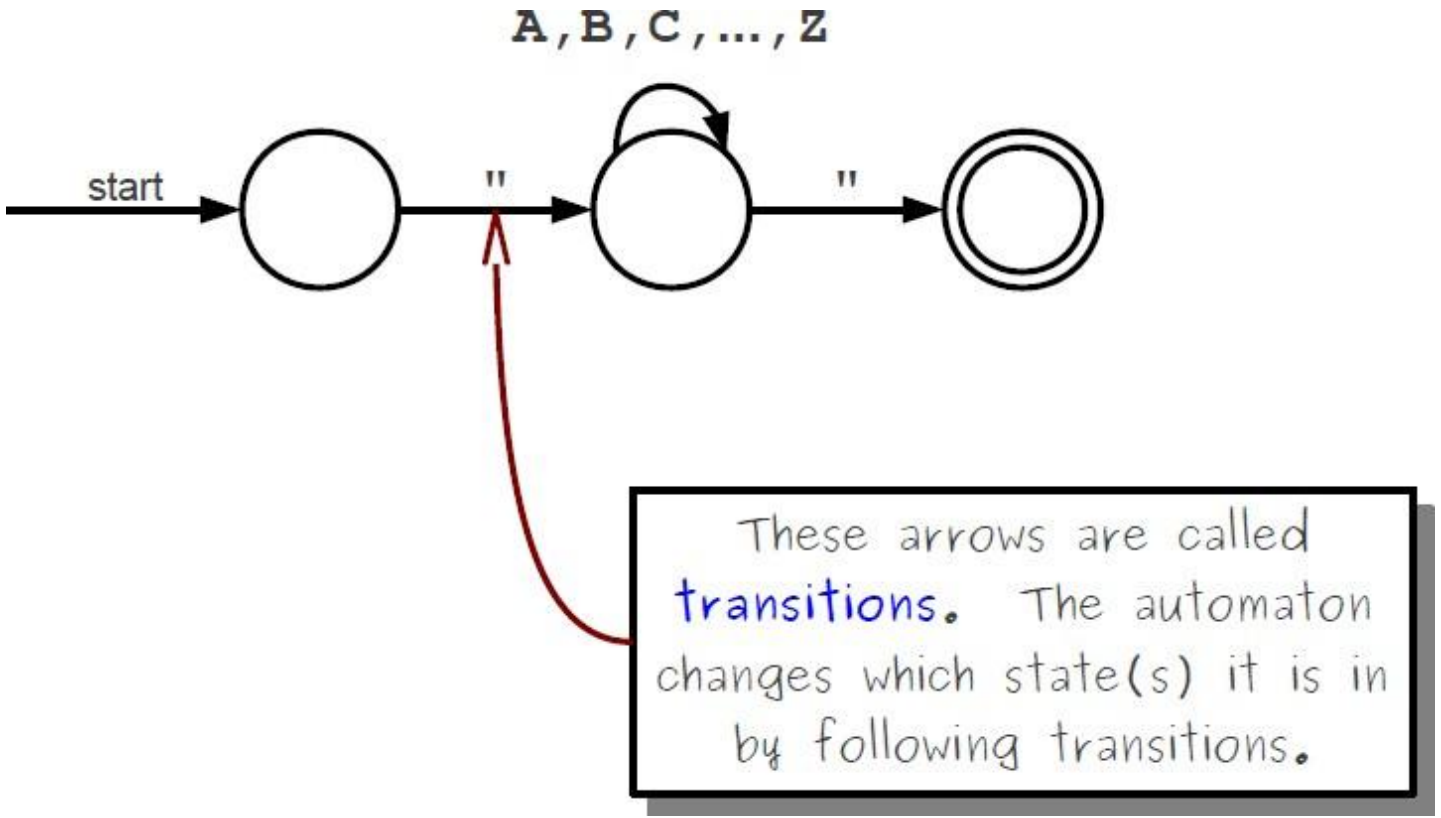


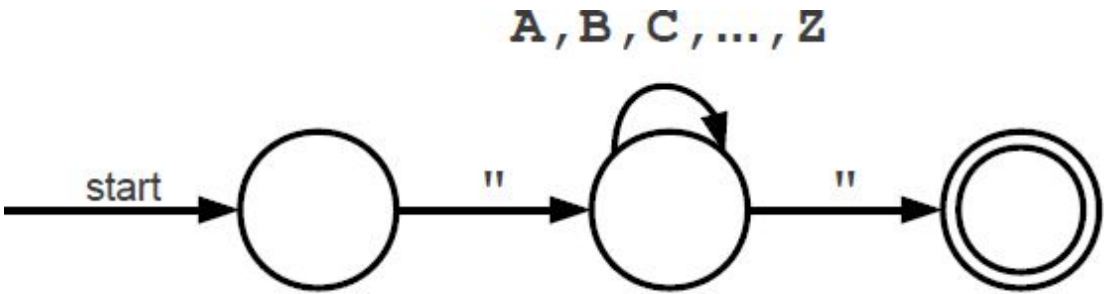
A SIMPLE AUTOMATON

جامعة عين شمس
AIN SHAMS UNIVERSITY
كلية الحاسبات والمعلومات
Faculty of Computer & Information Sciences

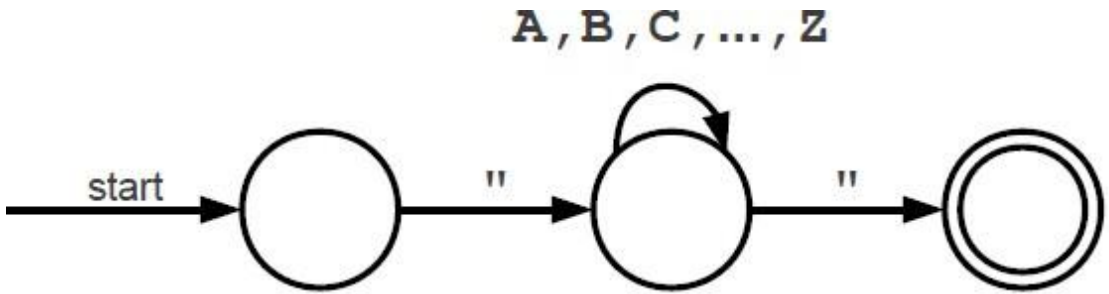

A , B , C , ... , Z
start
"
"

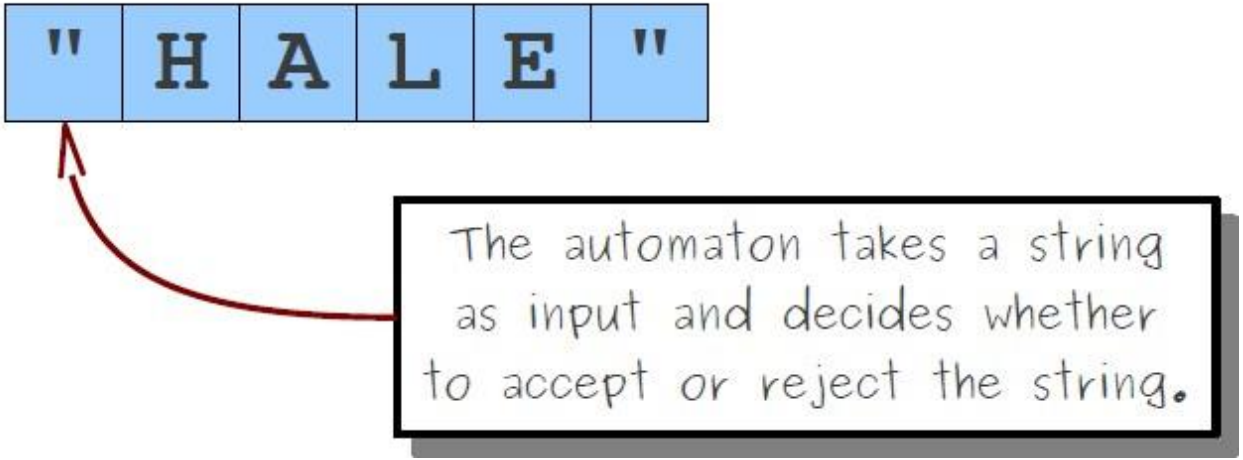

A,B,C,...,Z
start
"
"
Each circle is a state of the automaton. The automaton's configuration is determined by what state(s) it is in.

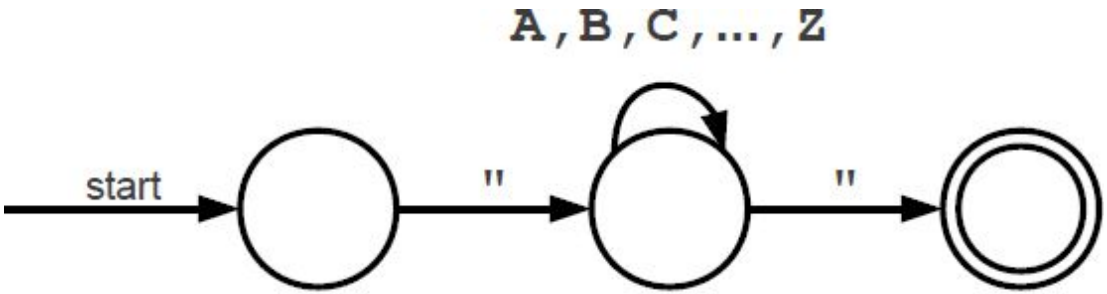

A, B, C, ..., Z
start
"
"
These arrows are called transitions. The automaton changes which state(s) it is in by following transitions.



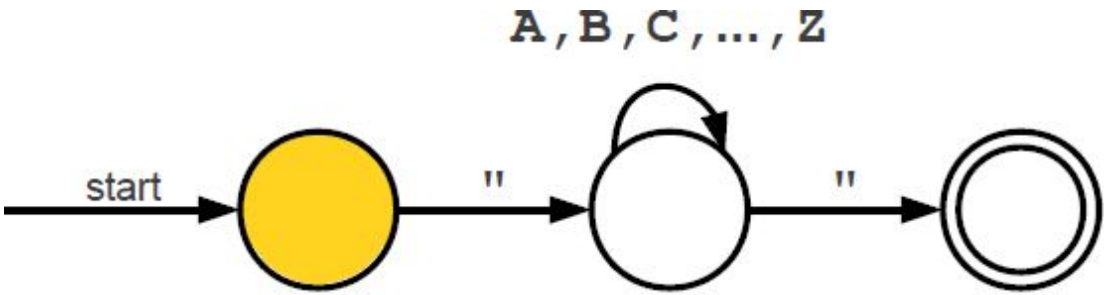
"	H	A	L	E	"


A, B, C, ..., Z
start
"
"

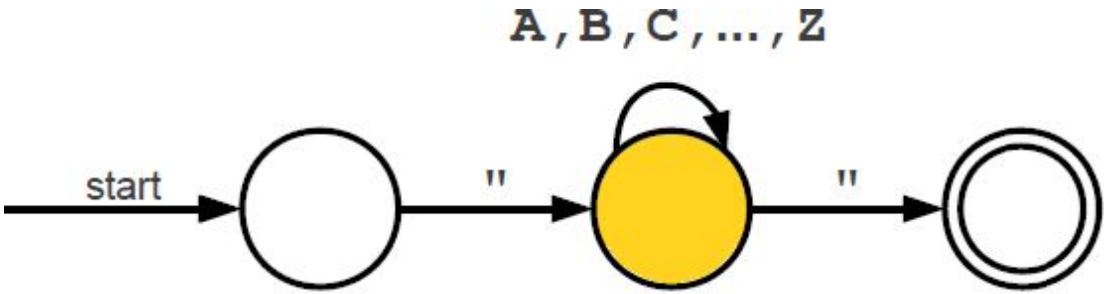

" H A L E "
The automaton takes a string as input and decides whether to accept or reject the string.

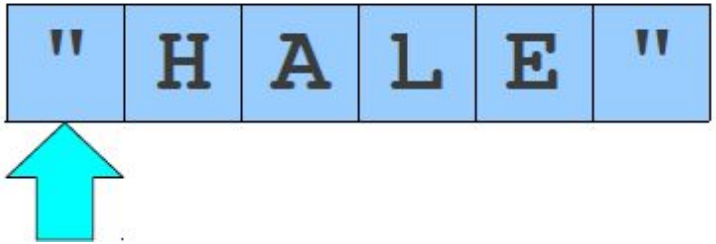

A, B, C, ..., Z
start
"
"

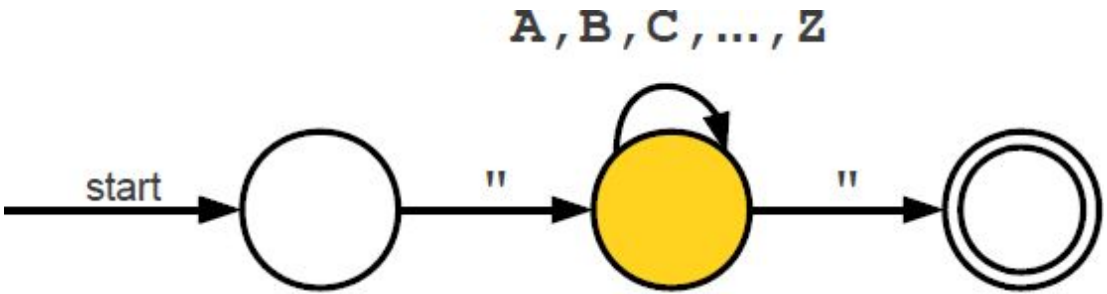

"
H
A
L
E
"


A , B , C , … , Z
start
"
"

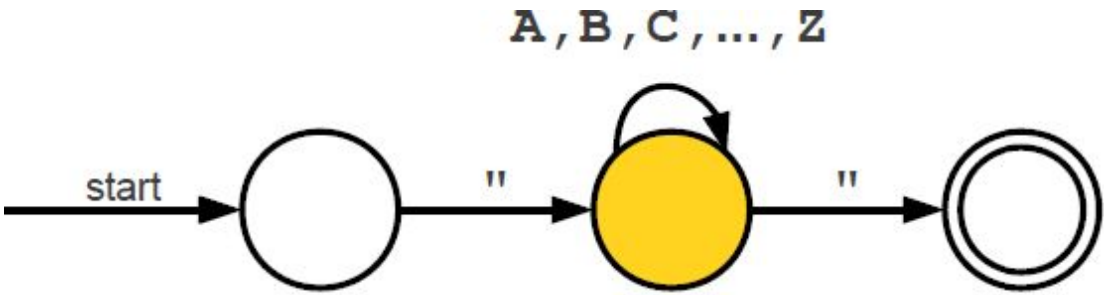

"
H
A
L
E
"

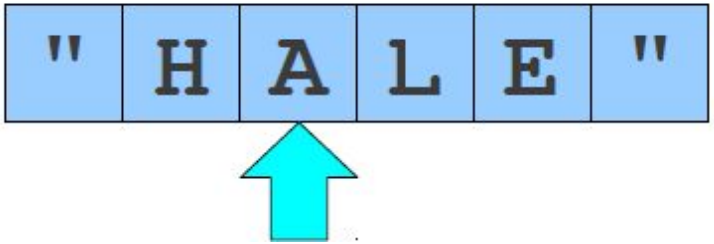

A , B , C , … , Z
start
"
"

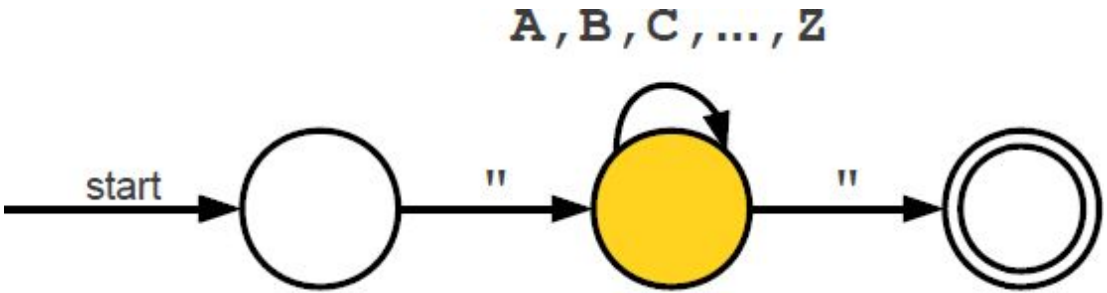

"
H
A
L
E
"

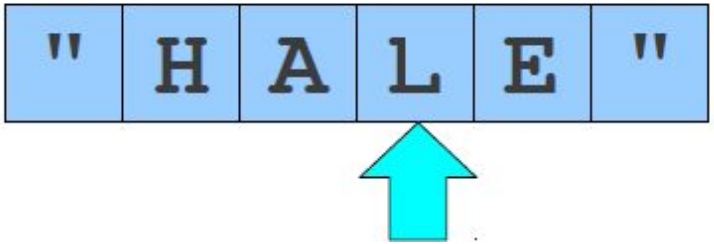

A , B , C , … , Z
start
"
"

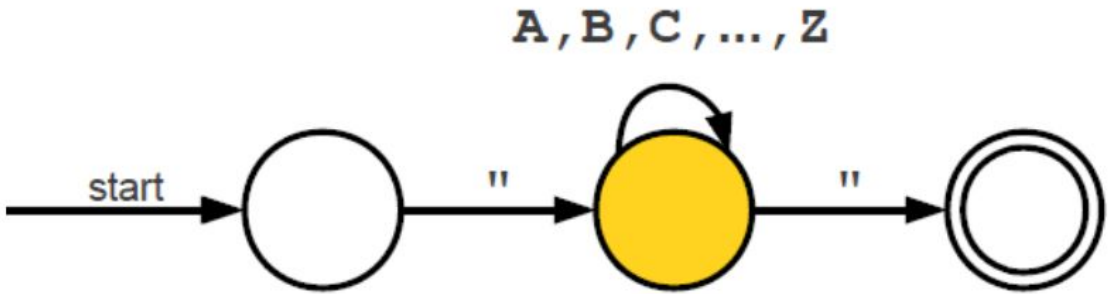

"
H
A
L
E
"

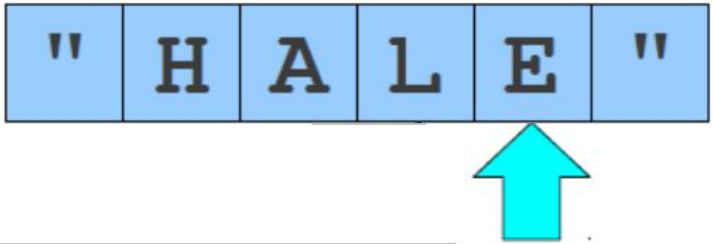

A , B , C , … , Z
start
"
"

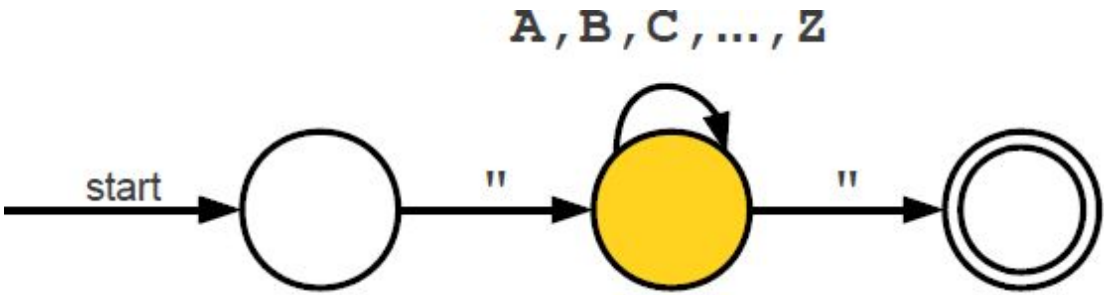

"
H
A
L
E
"

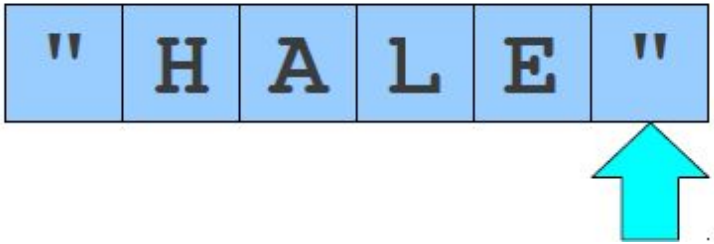

A , B , C , … , Z
start
"
"

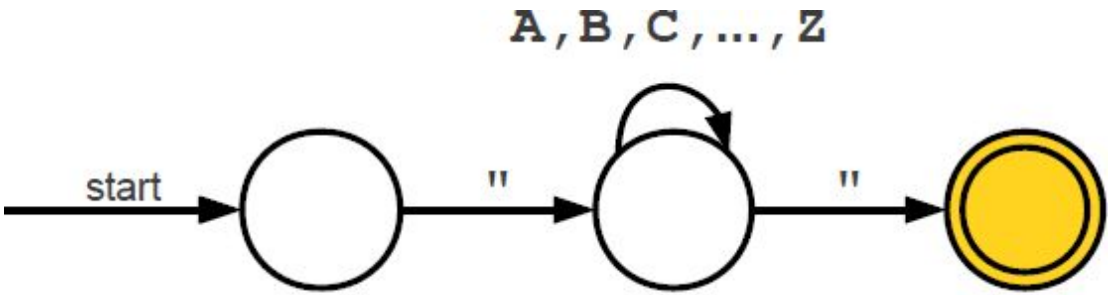

"
H
A
L
E
"

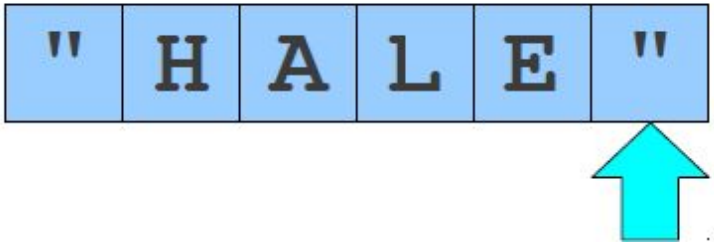

A, B, C, ..., Z
start
"
"

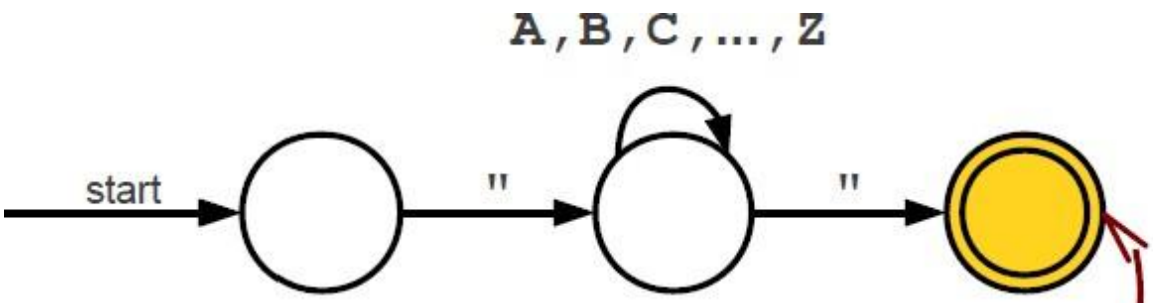

"
H
A
L
E
"


A , B , C , … , Z
start
"
"


"
H
A
L
E
"


A,B,C,...,Z
start
"
"

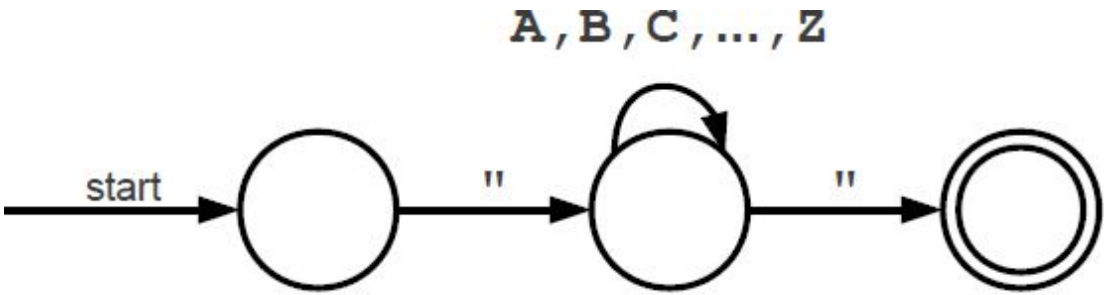

"
H
A
L
E
"



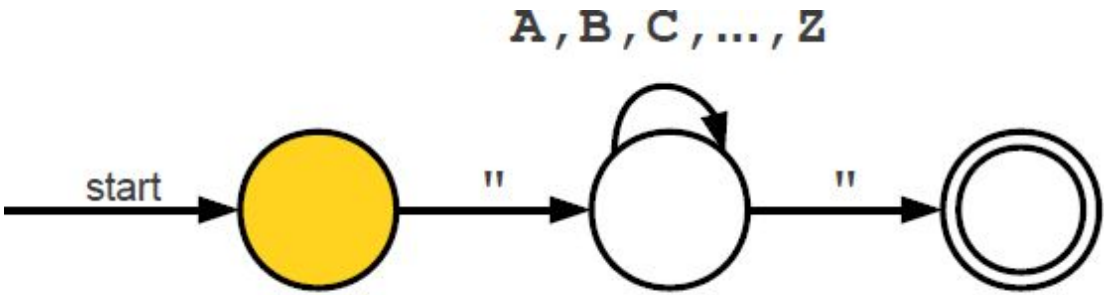
"	H	A	L	E	"

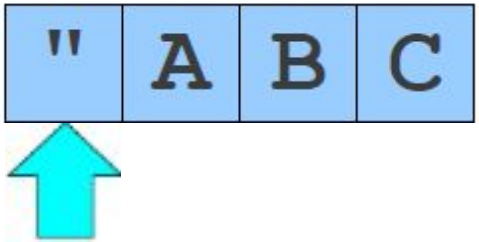
The double circle indicates that this state is an **accepting state.** The automaton accepts the string if it ends in an accepting state.

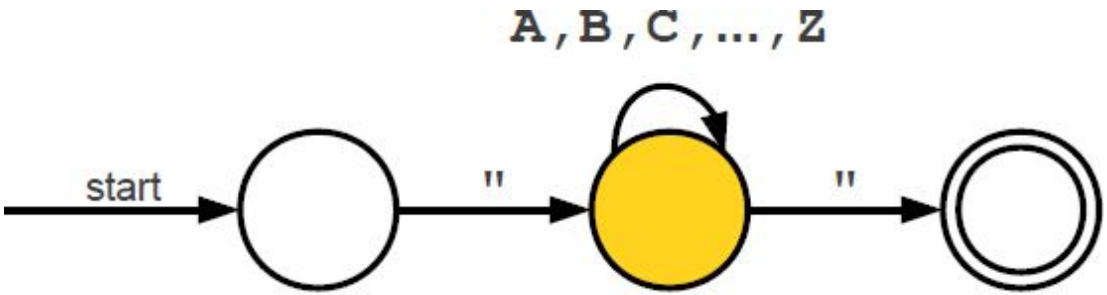
A SIMPLE AUTOMATON

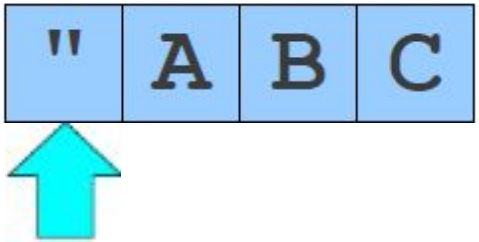


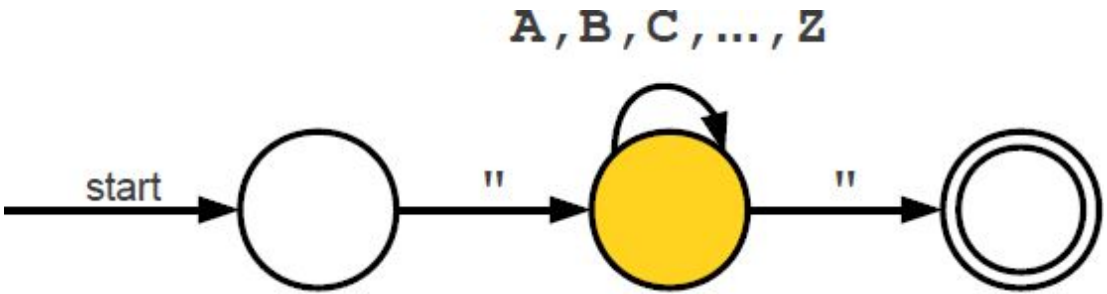
"	A	B	C

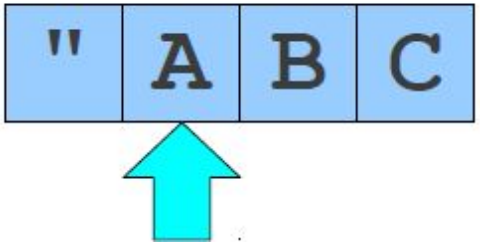

A,B,C,...,Z
start
"
"

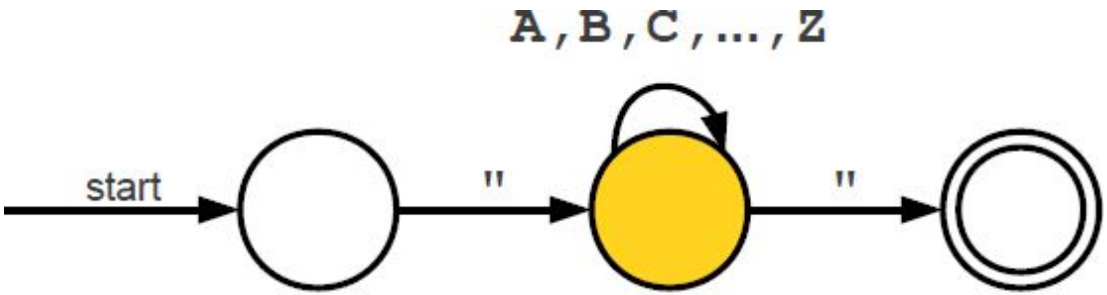

"
A
B
C

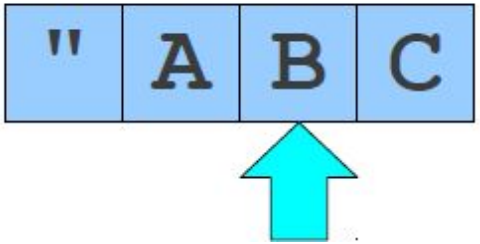

A,B,C,...,Z
start
"
"

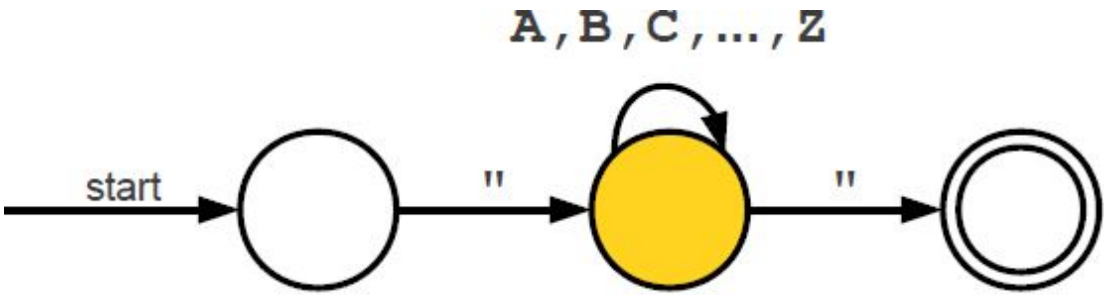

"
A
B
C

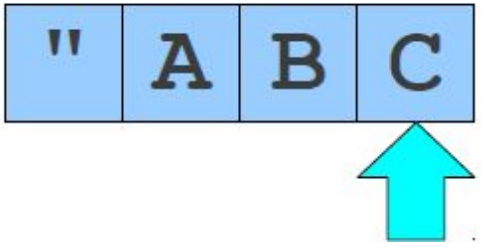

A,B,C,...,Z
start
"
"

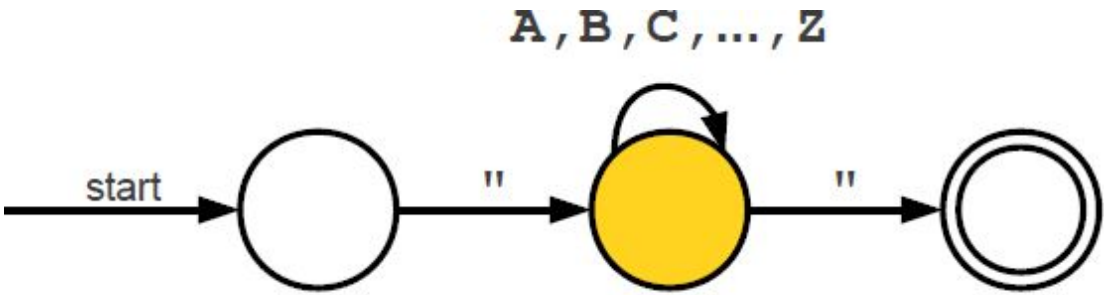

"
A
B
C


A, B, C, ..., Z
start
"
"

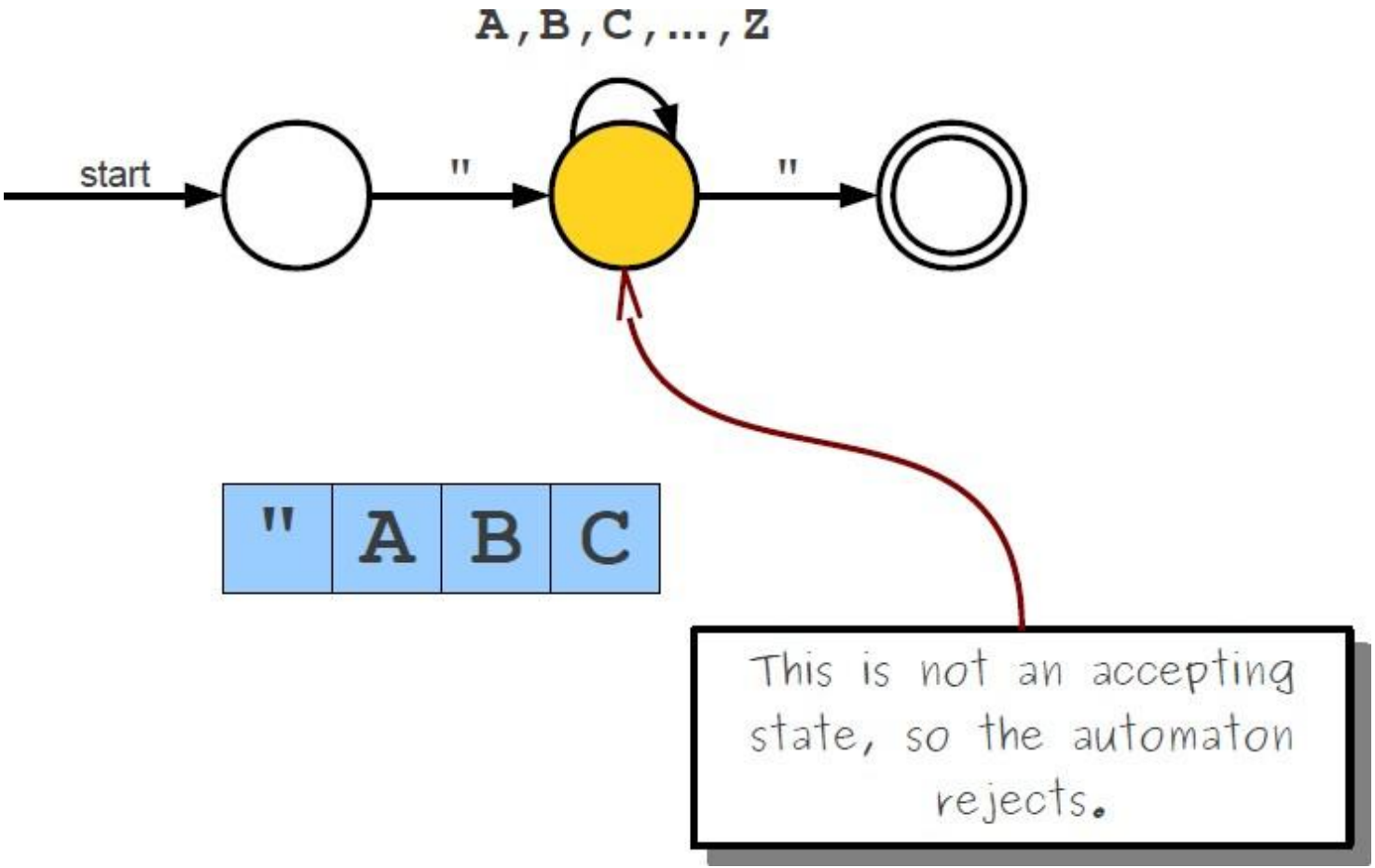

"
A
B
C


A, B, C, ..., Z
start
"
"

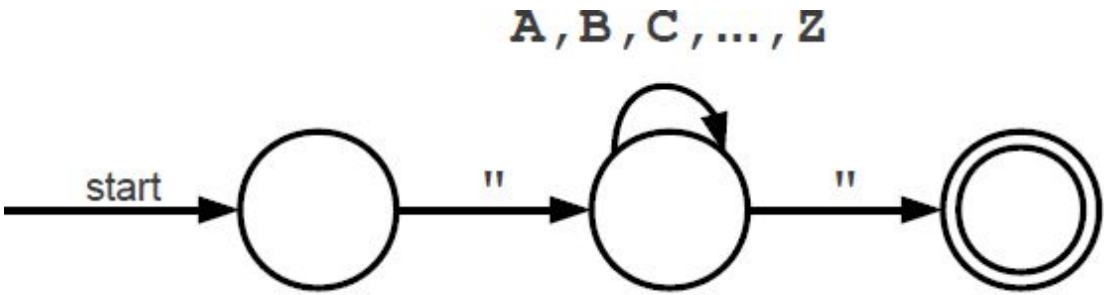

"
A
B
C

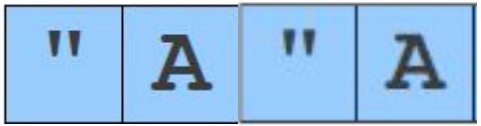


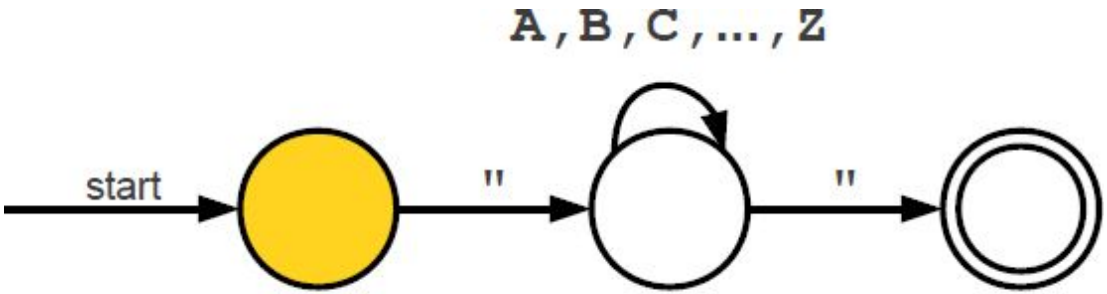
"	A	B	C

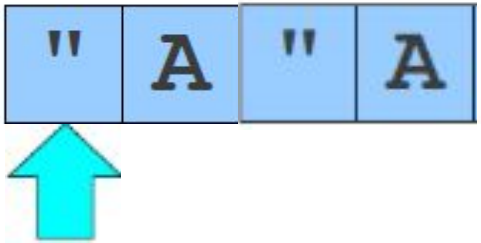

A,B,C,...,Z
start
"
"
" A B C
This is not an accepting state, so the automaton rejects.

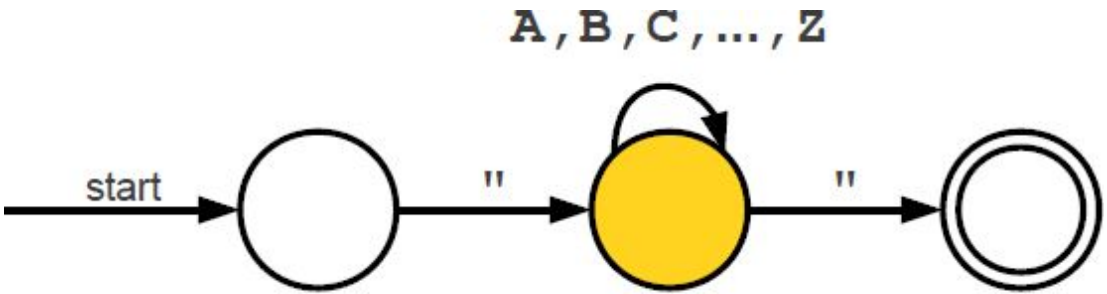
A SIMPLE AUTOMATON

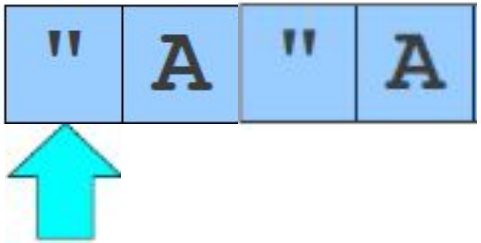

A,B,C,...,Z
start
"
"

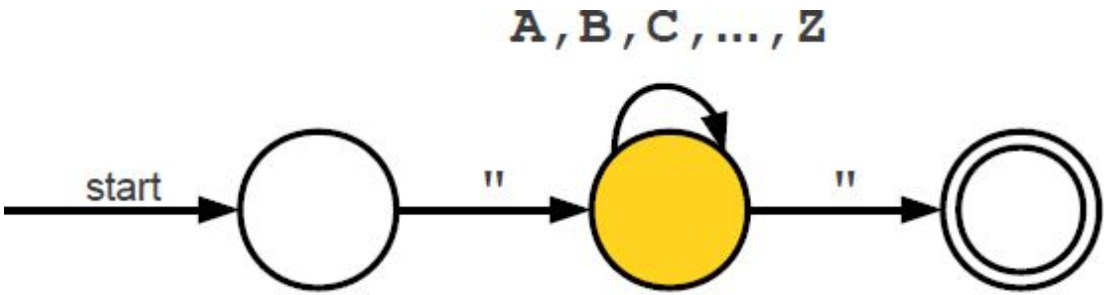

"
A
"
A

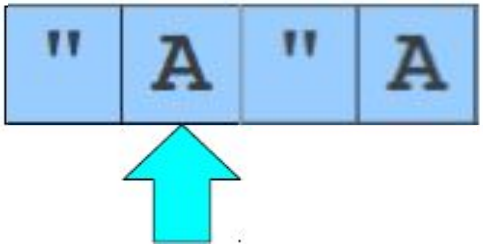

A, B, C, ..., Z
start
"
"

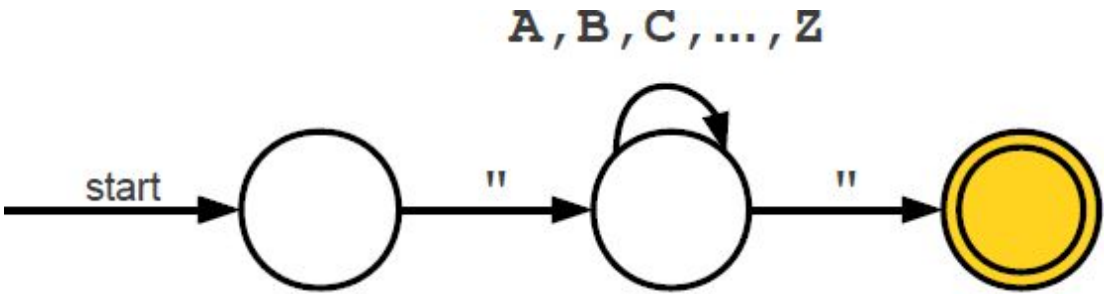

"
A
"
A

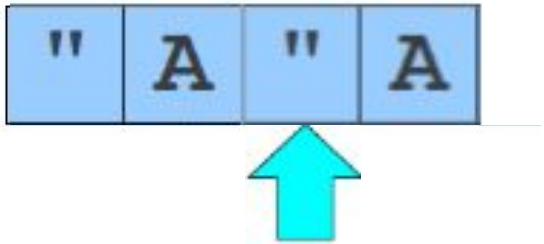

A, B, C, ... , Z
start
"
"

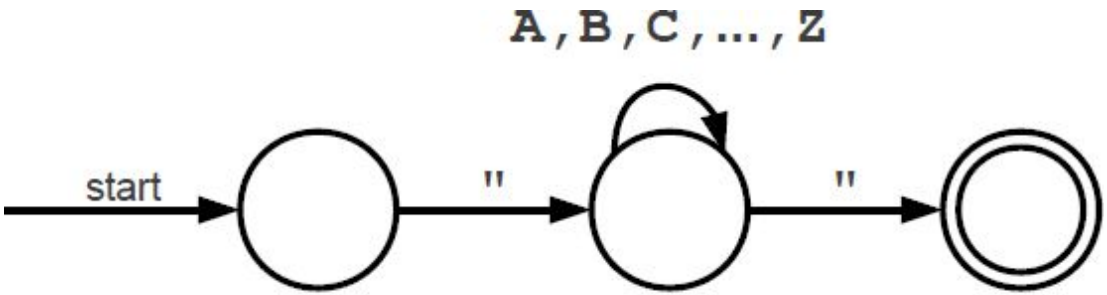

"
A
"
A

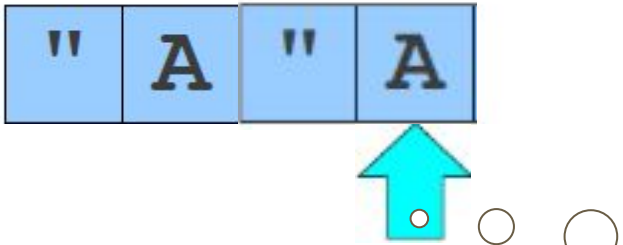

A, B, C, ..., Z
start
"
"


"
A
"
A


A , B , C , ... , Z
start
"
"

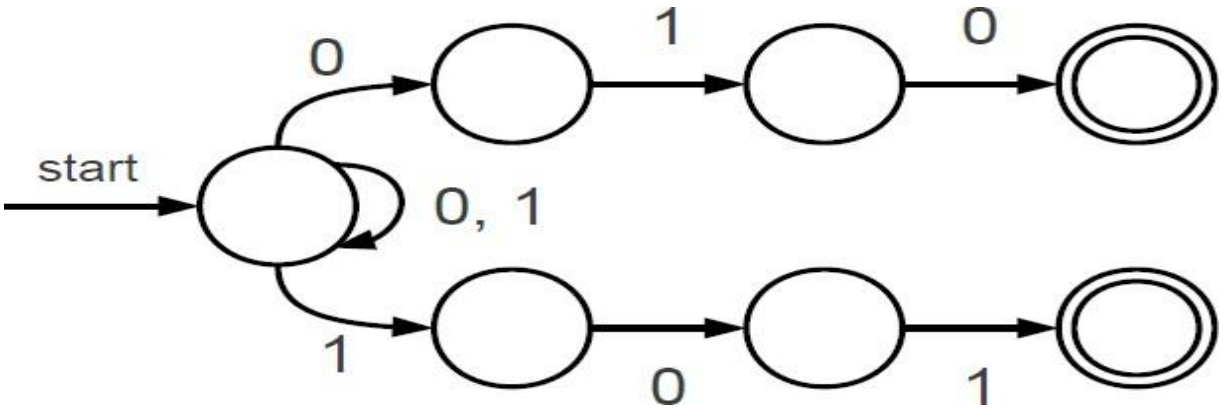

"
A
"
A



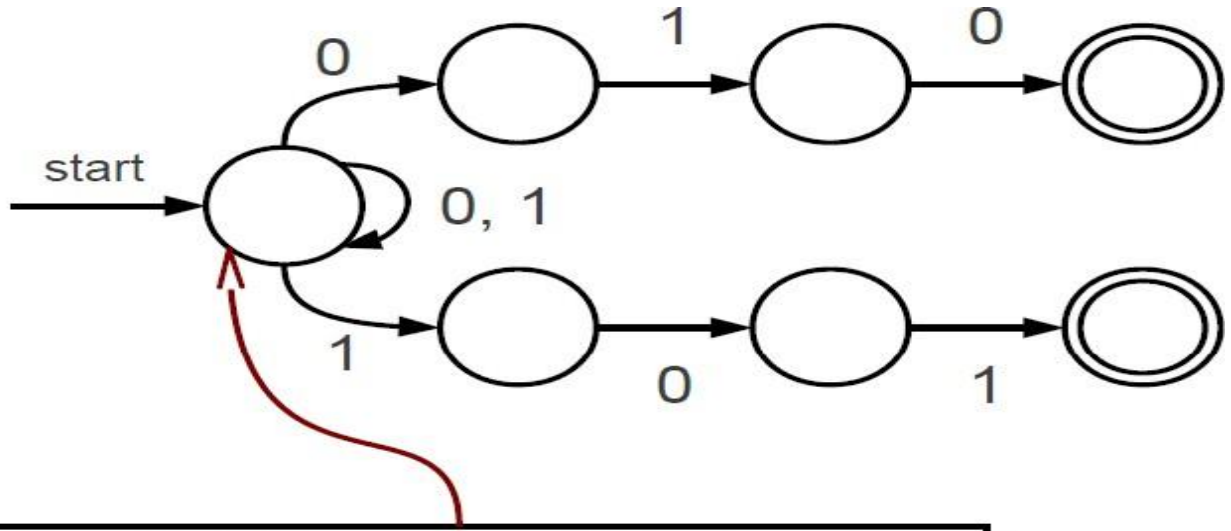


Rejected

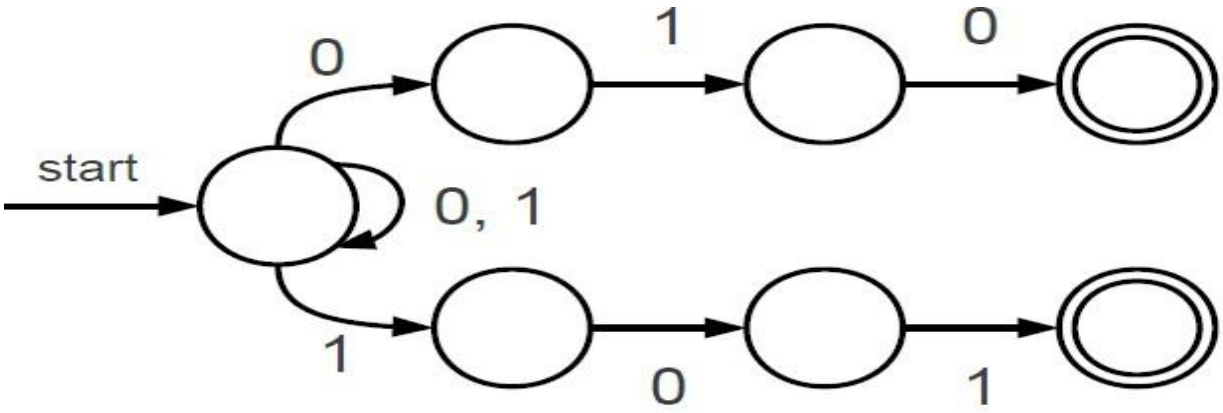
A SIMPLE AUTOMATON



A SIMPLE AUTOMATON

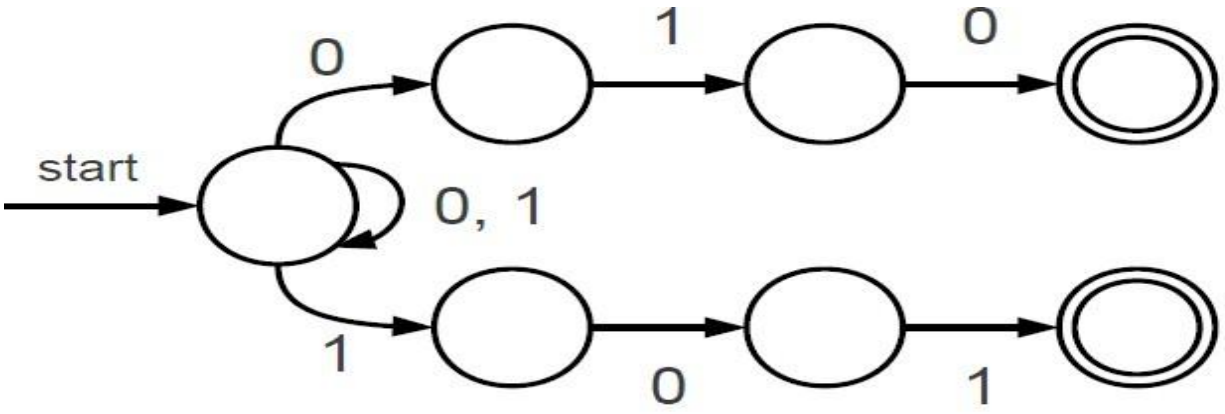


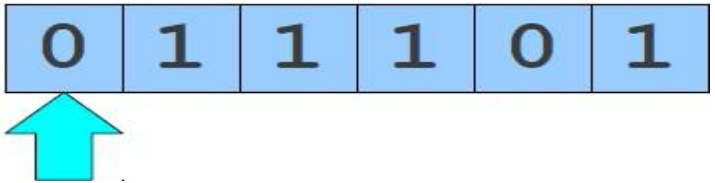
A SIMPLE AUTOMATON



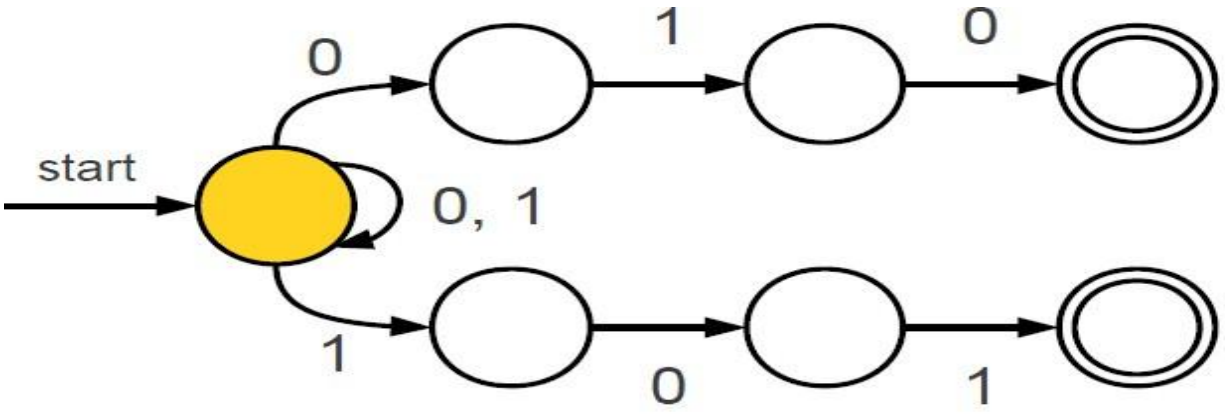
0	1	1	1	0	1

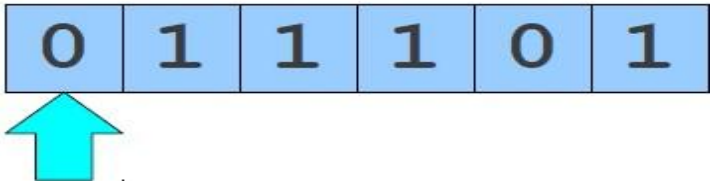
A SIMPLE AUTOMATON



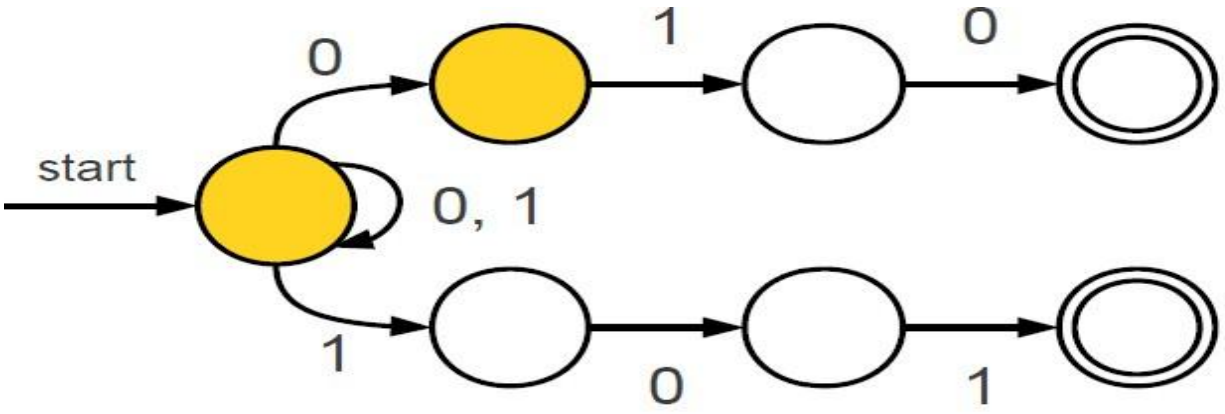


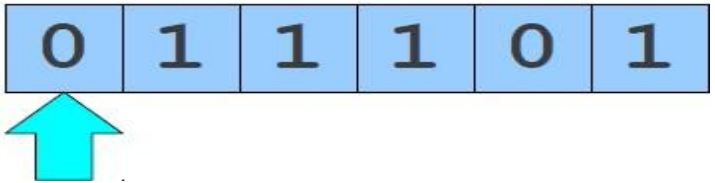
A SIMPLE AUTOMATON



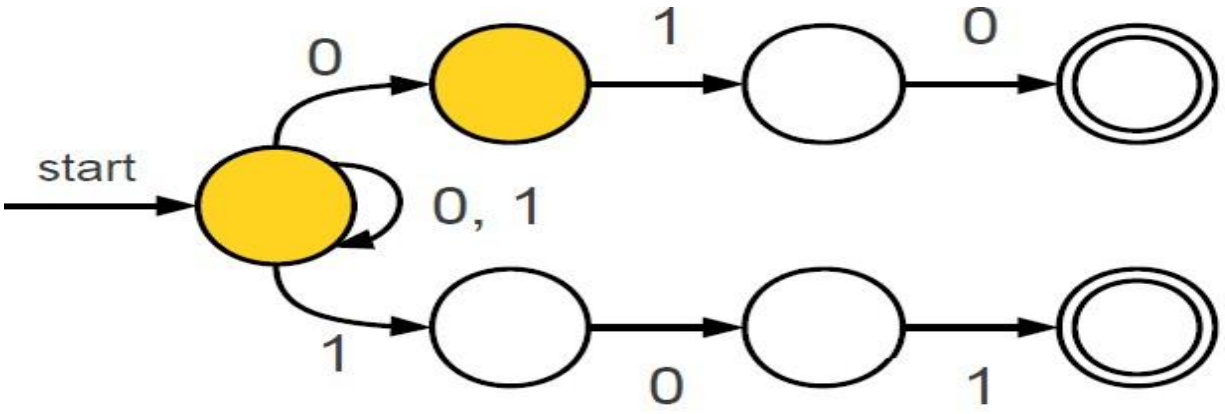


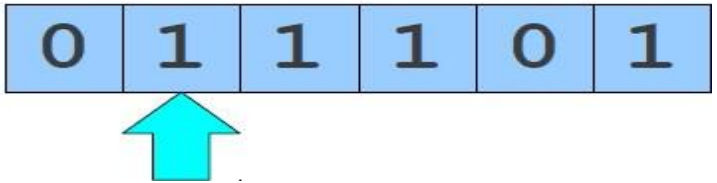
A SIMPLE AUTOMATON



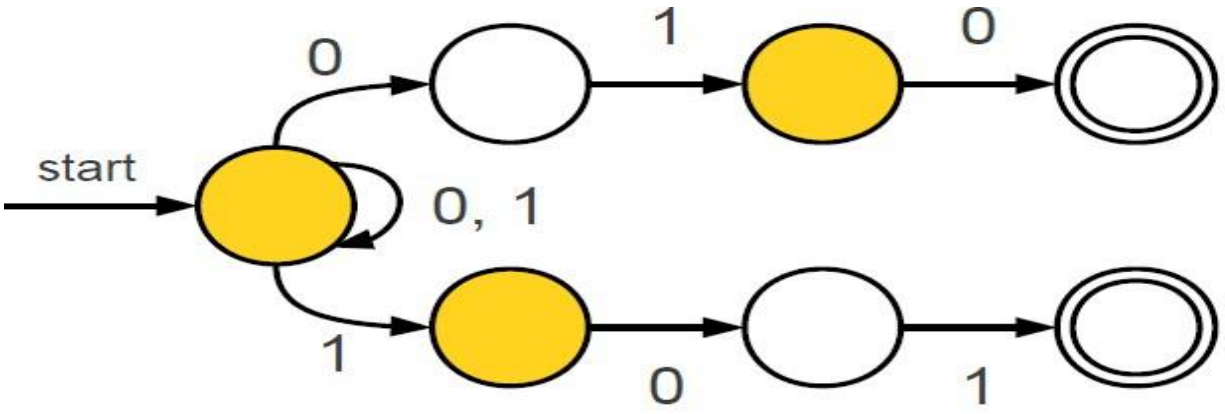


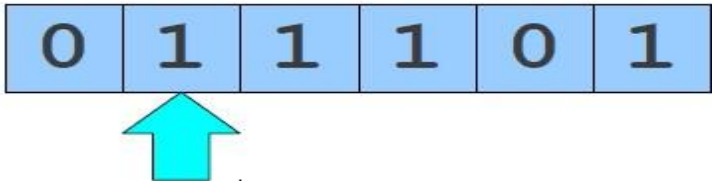
A SIMPLE AUTOMATON



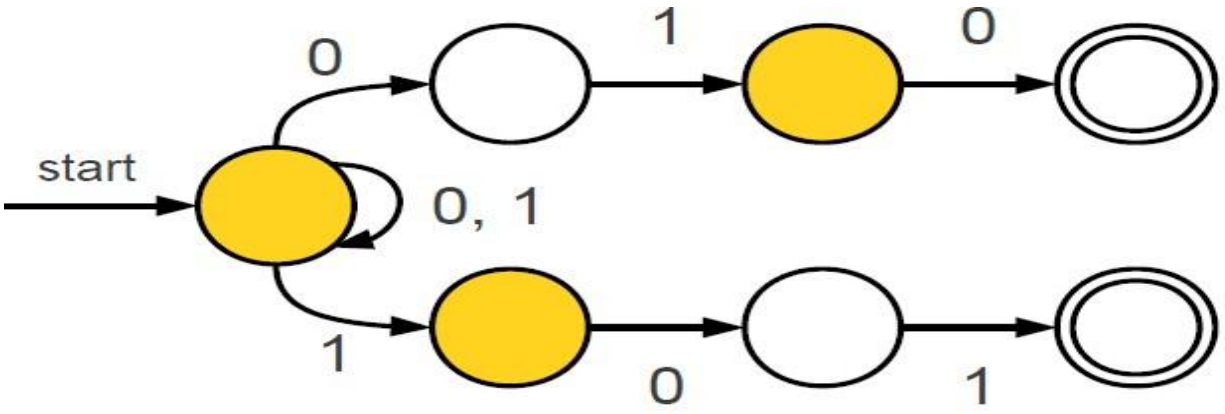


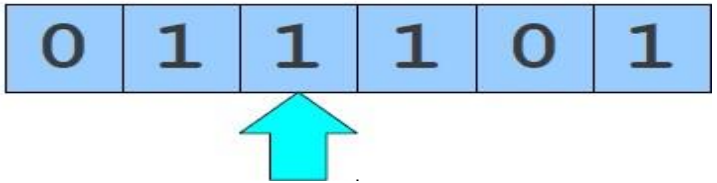
A SIMPLE AUTOMATON



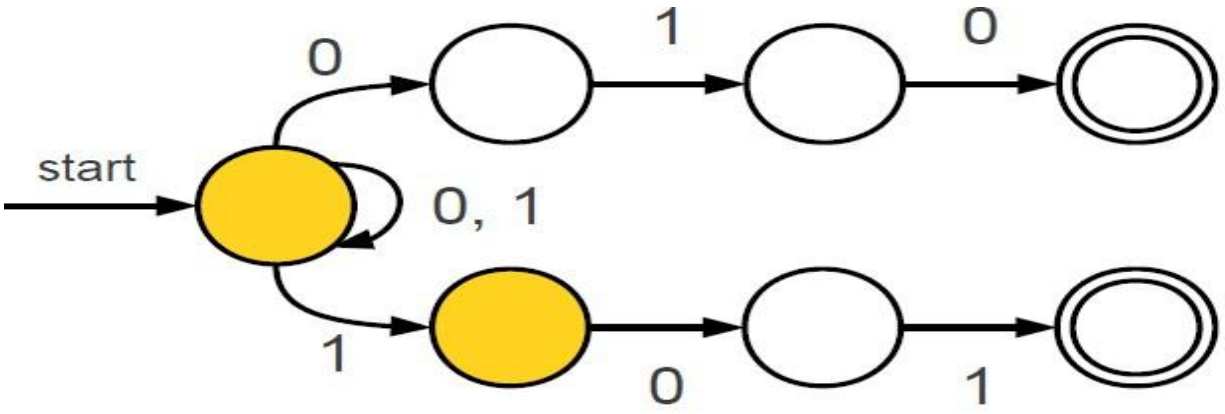


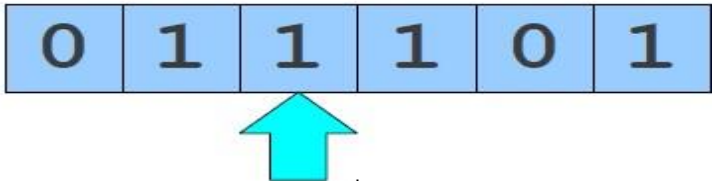
A SIMPLE AUTOMATON



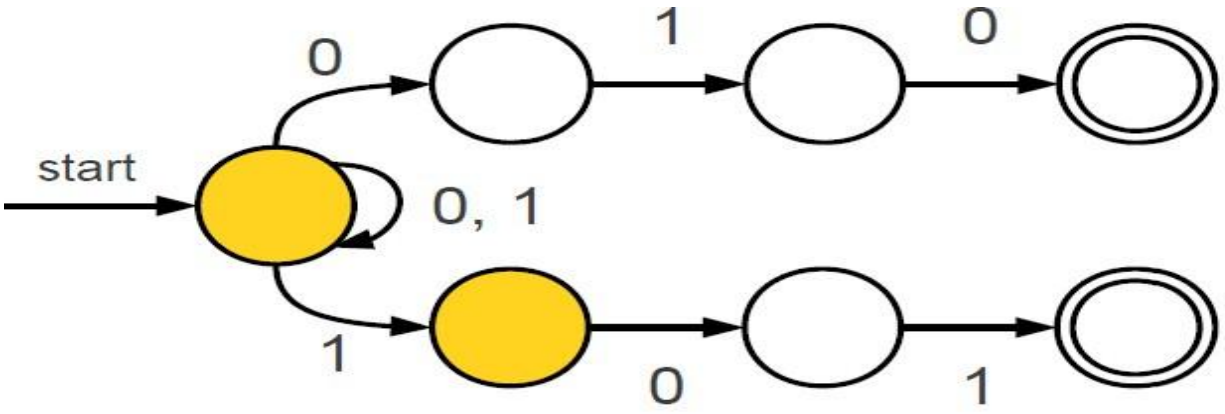


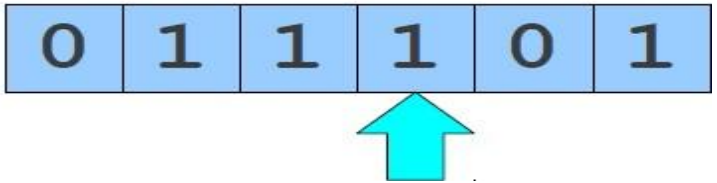
A SIMPLE AUTOMATON



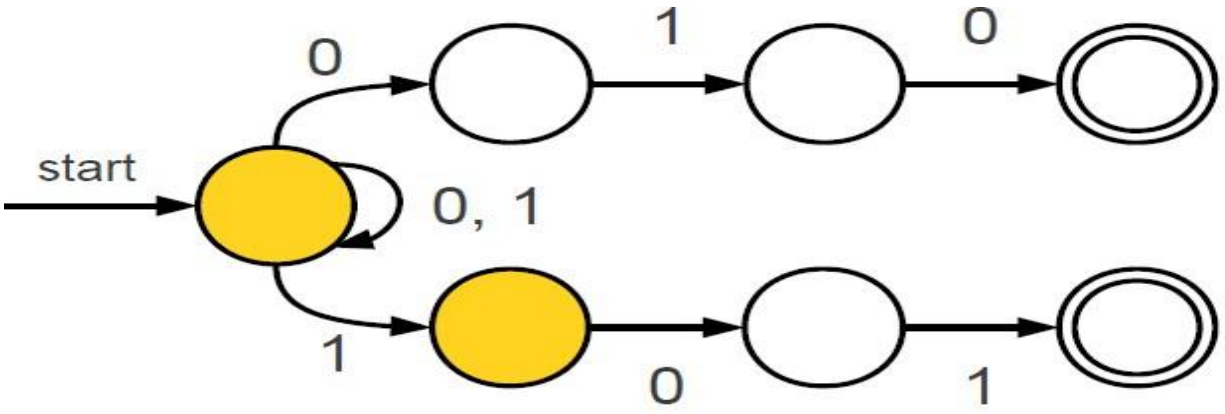


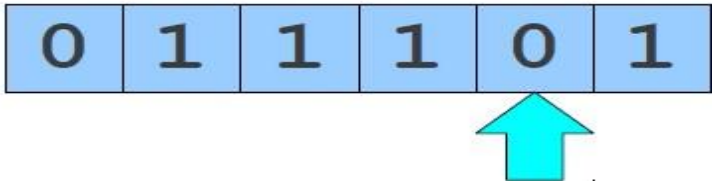
A SIMPLE AUTOMATON



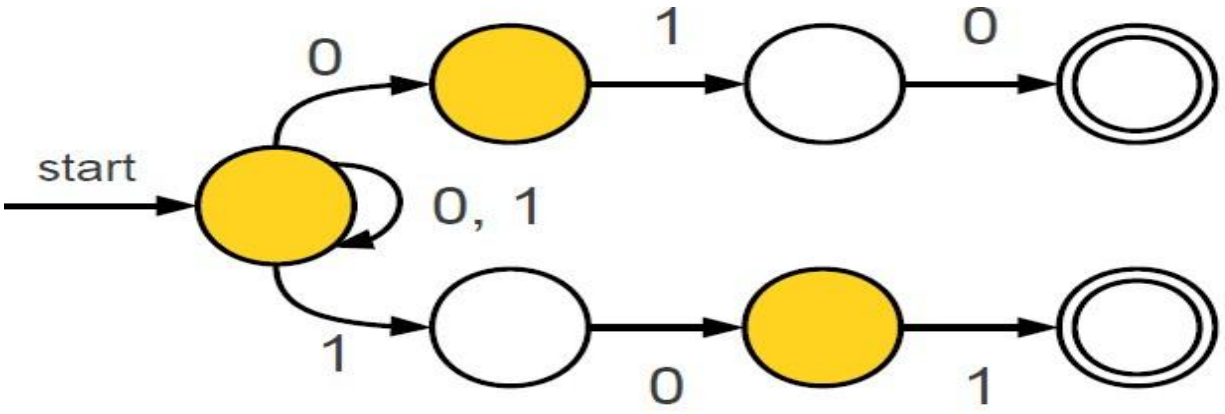


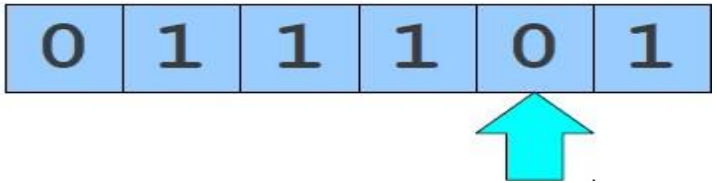
A SIMPLE AUTOMATON



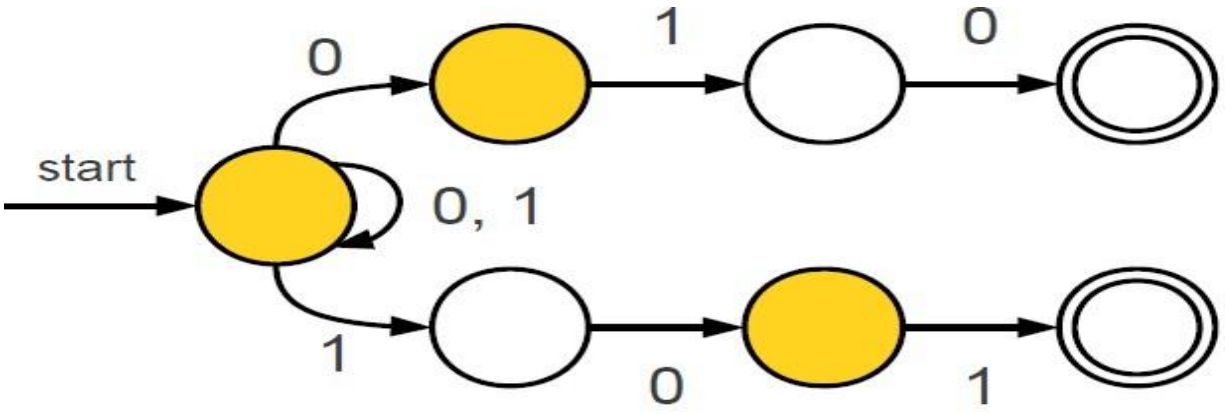


A SIMPLE AUTOMATON



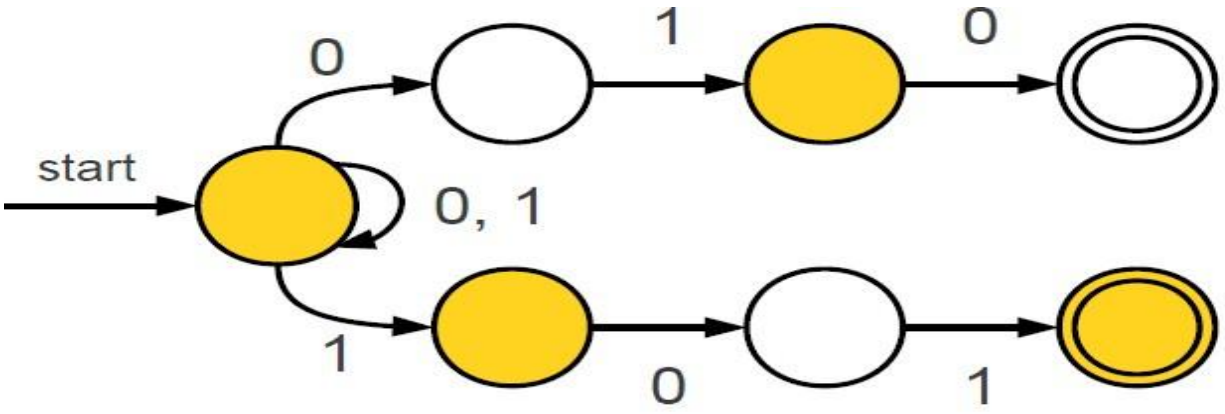


A SIMPLE AUTOMATON



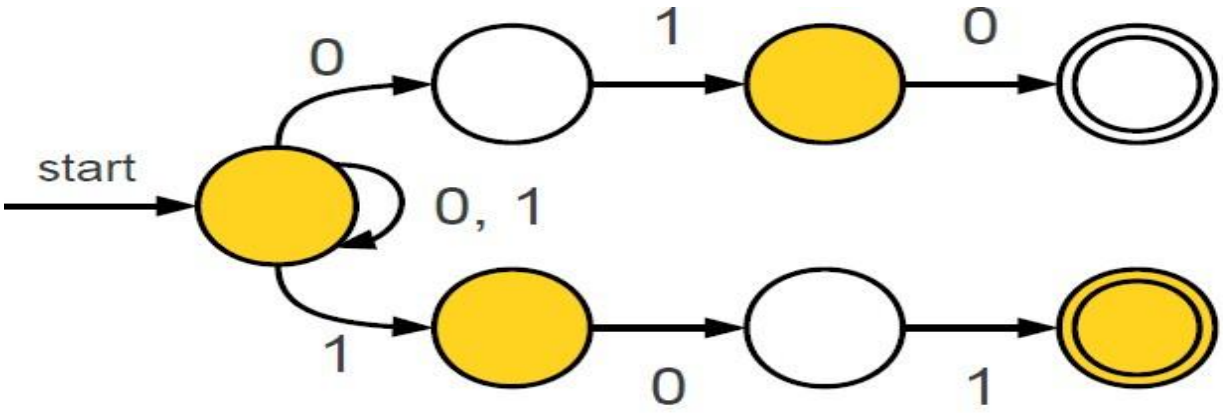


A SIMPLE AUTOMATON



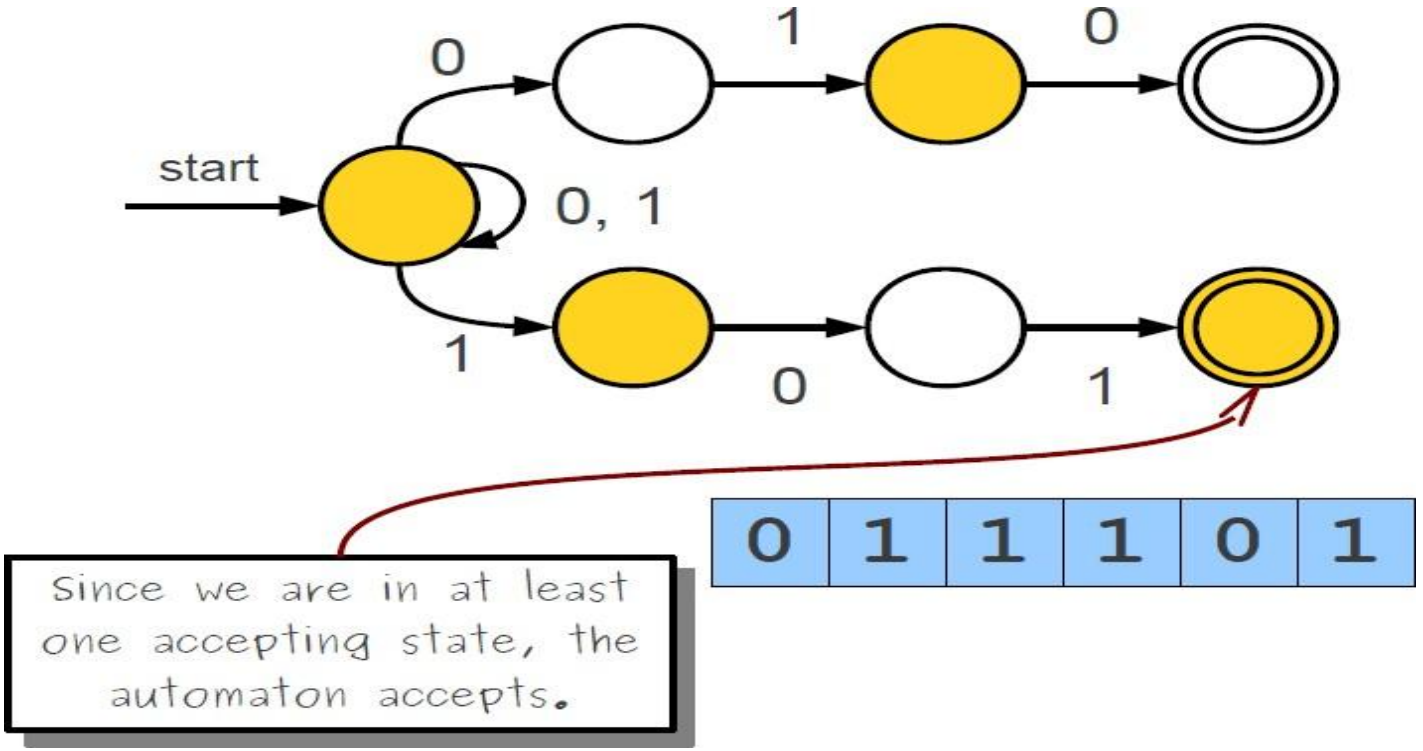


A SIMPLE AUTOMATON



0	1	1	1	0	1

A SIMPLE AUTOMATON




جامعة عين شمس
AIN SHAMS UNIVERSITY
كلية الحاسبات والمعلومات
Faculty of Computer & Information Sciences

DFA vs NFS

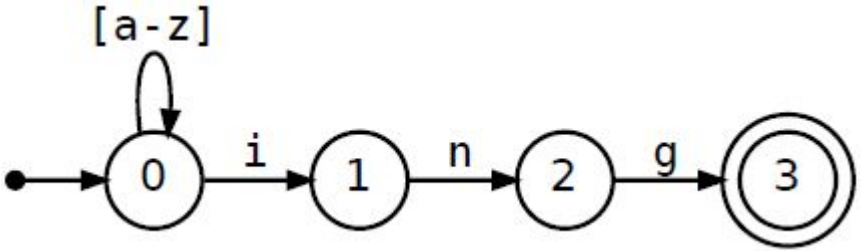
THE CONCEPT OF DFA



- A DFA is a special case of an FA where every state has no more than one outgoing edge for a given symbol.
- A DFA has no ambiguity:
  - For every combination of state and input symbol, there is exactly one choice of what to do next.
- A DFA is very easy to implement in software or hardware.
- Transition Matrix Representation (M[s, i]):
  - A 2D array where rows represent states, columns represent characters, and values represent the next state.
  - Empty transitions are marked as E(Error).

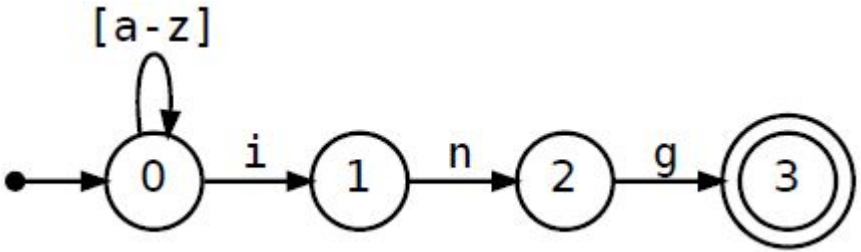
THE CONCEPT OF NFA

- An NFA permits multiple outgoing edges from a single state labeled with the same character, or contains **ε** (epsilon) transitions.
- Epsilon (**ε**) Transition: A free transition that can be traversed without consuming any characters from the input stream.
- The Problem: Ambiguous execution paths.
- Example RE: [a-z]*ing (all lowercase words ending in "ing")



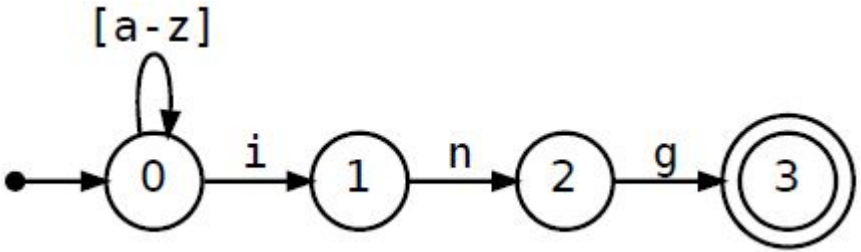
Tracing the Ambiguity of an NFA

- Case Study Input: The string "sing" on the [a-z]*ing NFA.
- Path A (Incorrect Guess):
  - State 0 —s—> State 0
  - State 0 —i—> State 0
  - State 0 —n—> State 0
  - State 0 —g—> State 0 ⇒ REJECT (State 0 is not accepting)



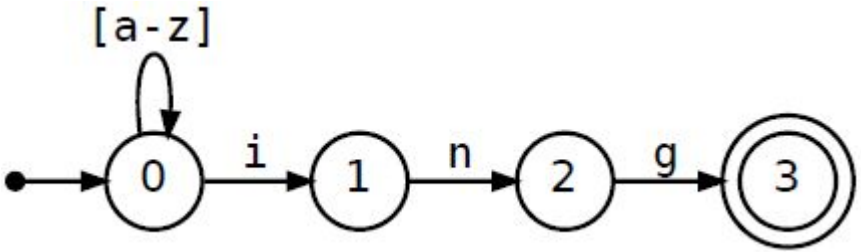
Tracing the Ambiguity of an NFA

- Case Study Input: The string "sing" on the [a-z]*ing NFA.
- Path B (Correct Guess):
  - State 0 —s—> State 0
  - State 0 —i—> State 1
  - State 1 —n—> State 2
  - State 2 —g—> State 3 ⇒ ACCEPT(State 3 is accepting)



Tracing the Ambiguity of an NFA

- Conclusion:
  - The machine must explore all paths.
  - How can we model this in software?

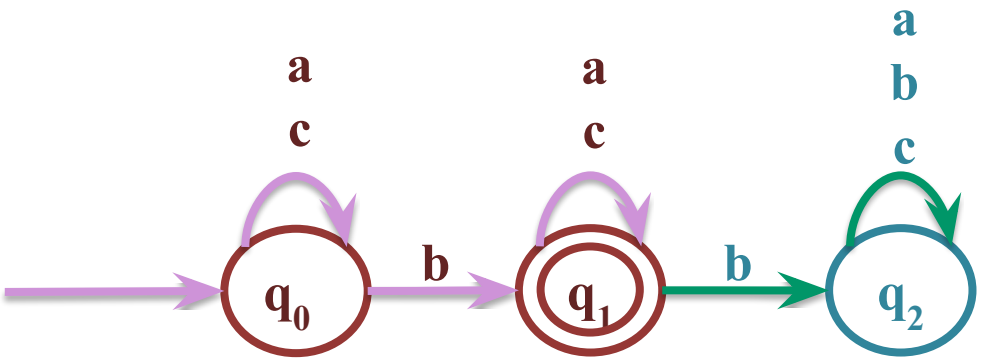




Deterministic Finite Automata (DFA)

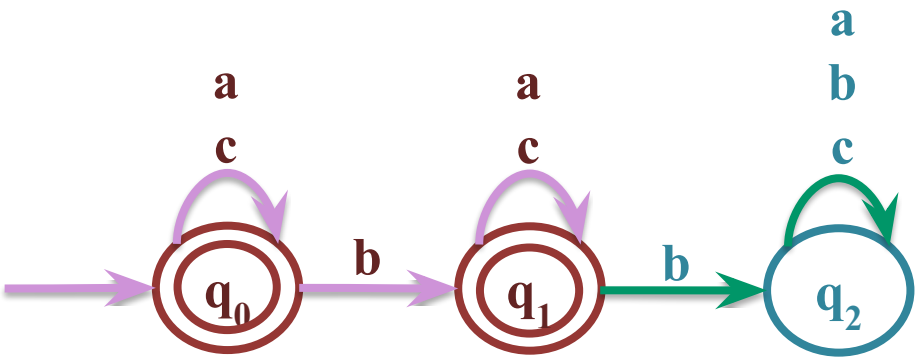
Example 1

- For Σ = {a, b, c}, construct DFA that accepts the sets of all strings with exactly one b.



Example 2

- For Σ = {a, b, c}, construct DFA that accepts the sets of all strings with at most one b.



Example 3

- Construct DFA that accepts the sets of all numbers, e.g. 2, -4, +10, 5.9, -1.4 E -10, etc.

Number → SignedNat(\.Natural)?(E SignedNat)?

Number → SignedNat(\.Natural)? (E SignedNat)?

SignedNat → (+ | -)? Natural

Natural → $Digit^{+}$

Digit → [0–9]

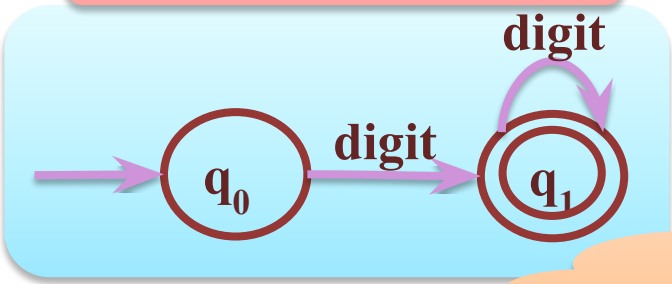
Starting from bottom to up To design the DFA

Example 3

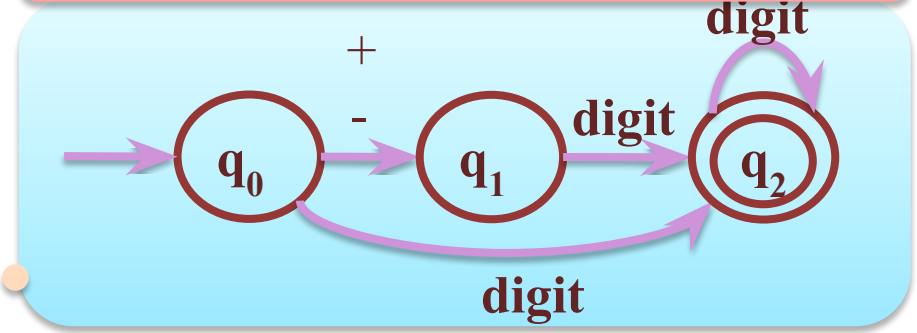
Number → SignedNat(\.Natural)? (E SignedNat)?

Digit → [0 – 9]

Natural = Digit +



SignedNat = (+ | -)? Natural

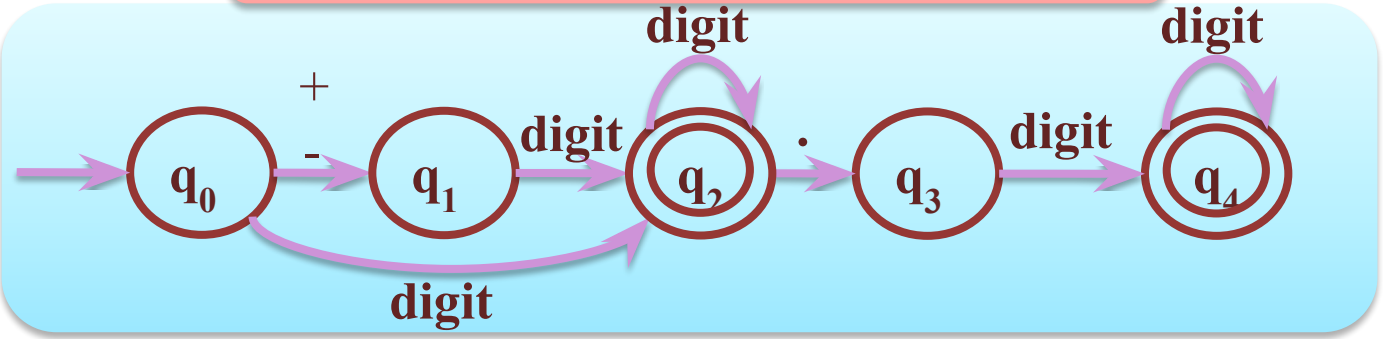


Starting from bottom to up

Example 3

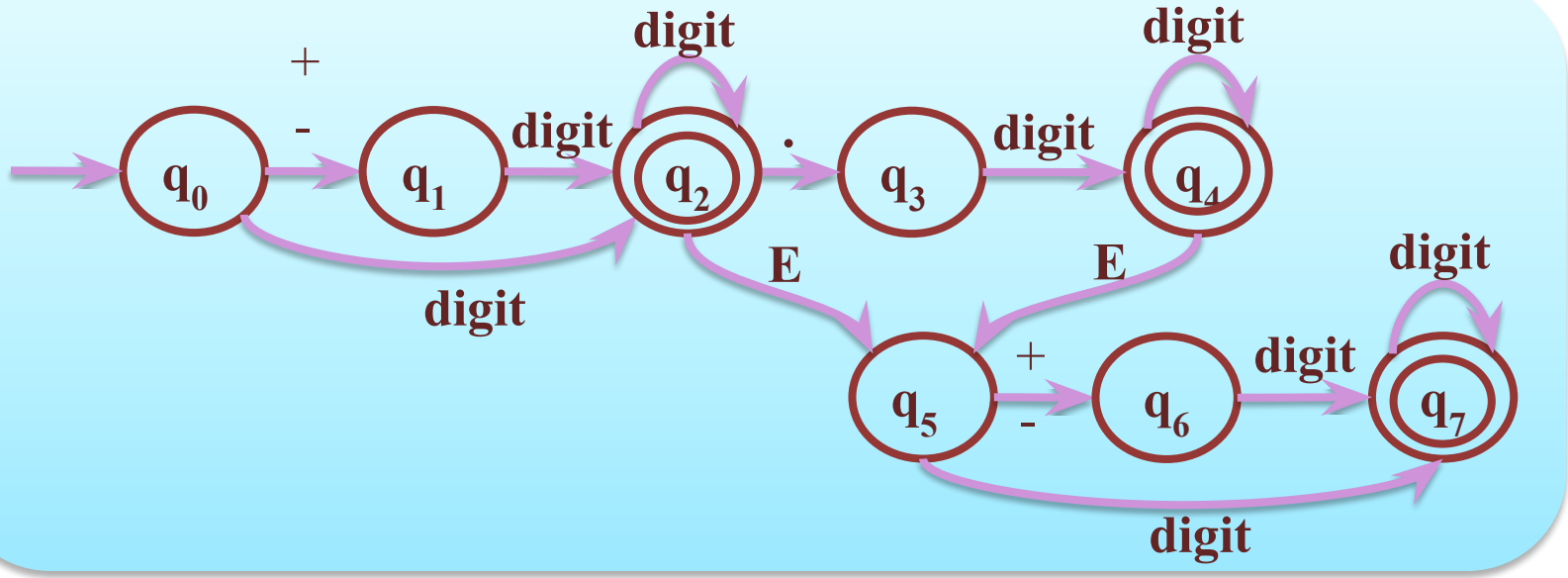
Number → SignedNat(\.Natural)? (E SignedNat)?

SignedNat(\.Natural)?



Example 3

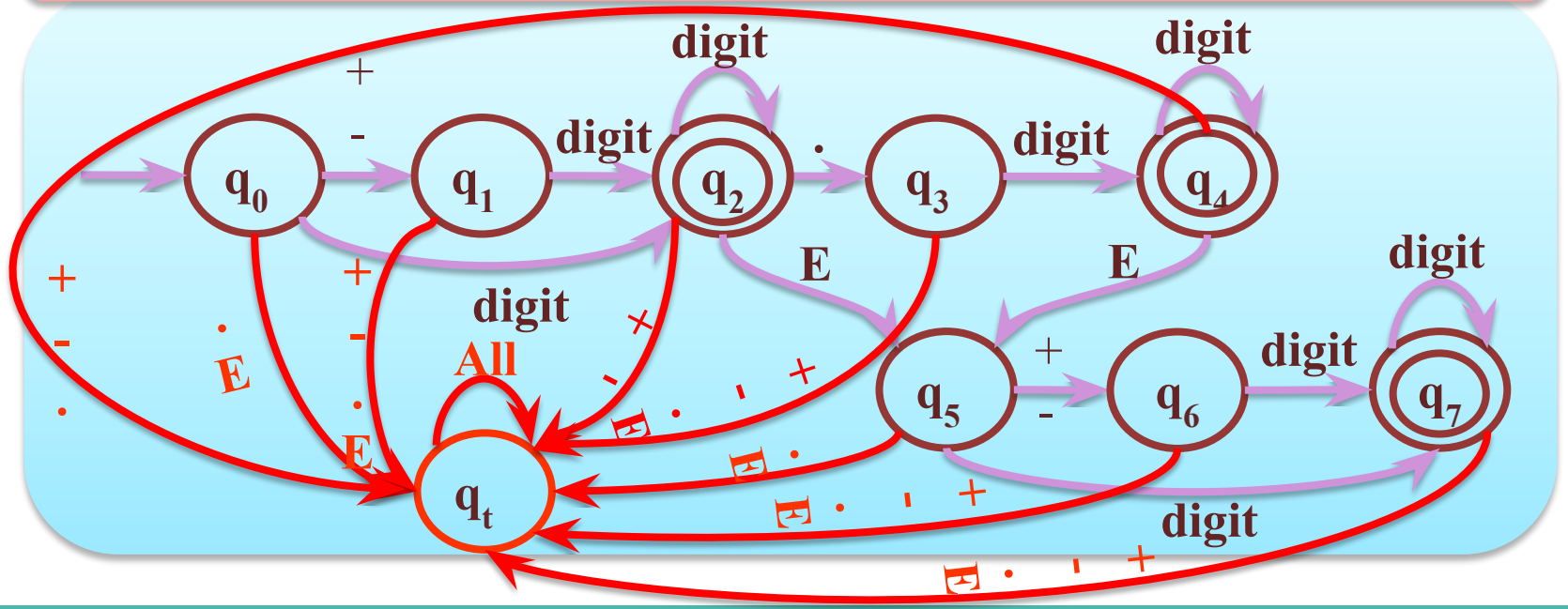
Number → SignedNat(\.Natural)? (E SignedNat)?



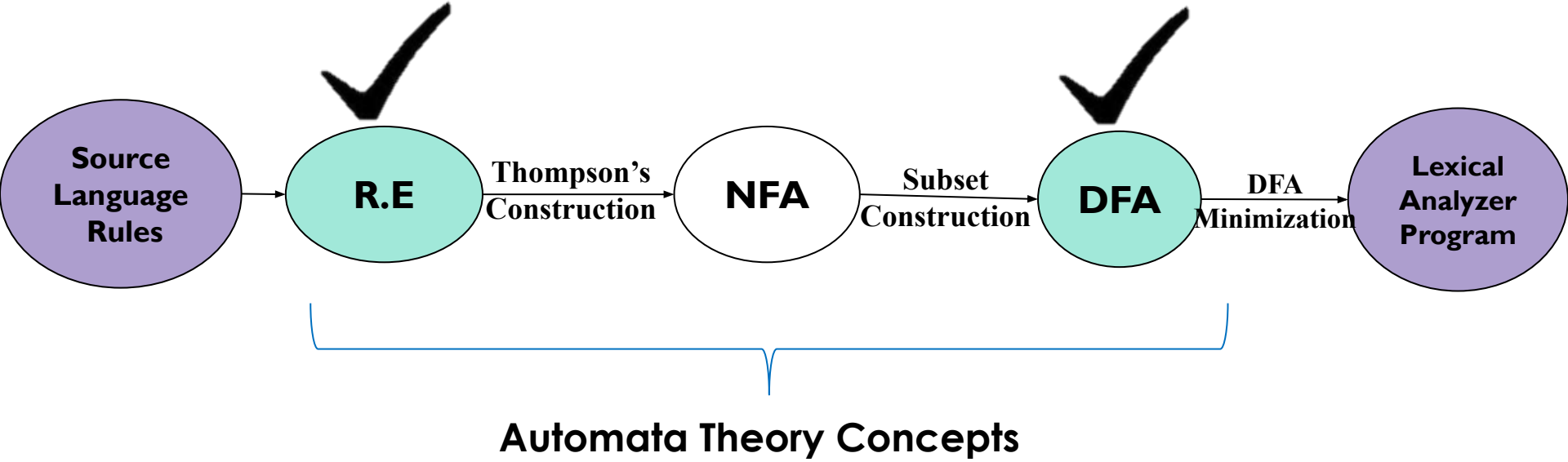
Example 3

جامعة عين شمس
AIN SHAMS UNIVERSITY
كلية الحاسبات والمعلومات
Faculty of Computer & Information Sciences

Number → SignedNat(\.Natural)? (E SignedNat)?



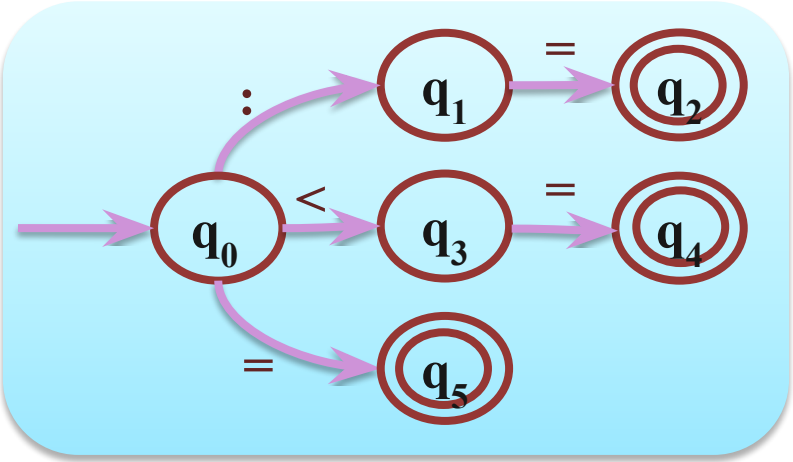
Lexical Analyzer and Automata



NonDeterministic Finite Automata (NFA)

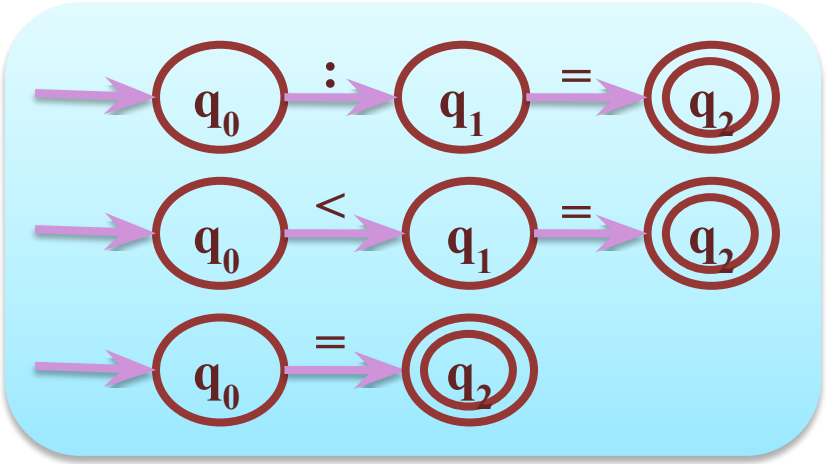
Example

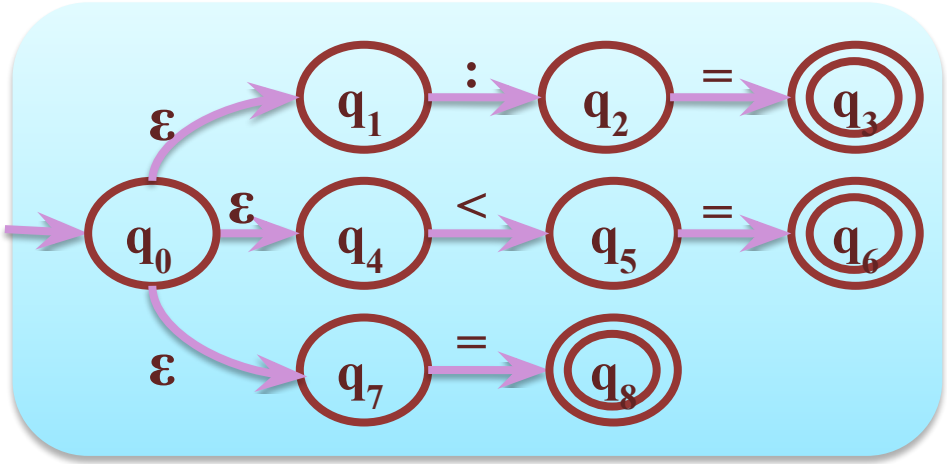
- Construct NFA that accepts the sets of all the following tokens: **:=**, **<=**, and **=**



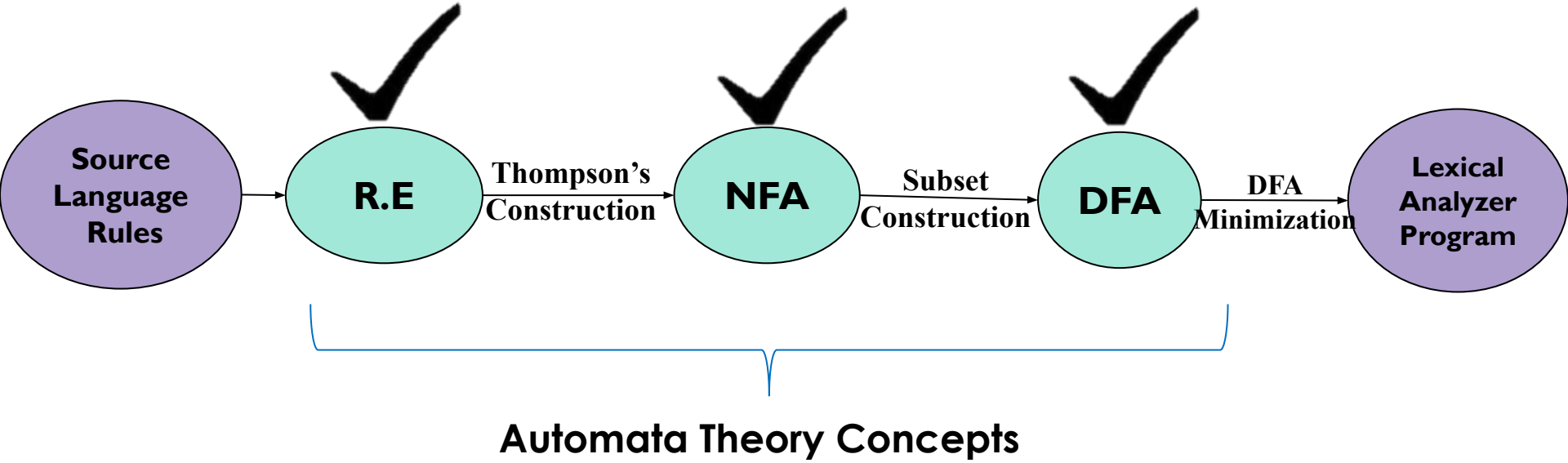
Example

- Construct NFA that accepts the sets of all the following tokens: **:=**, **<=**, and **=**





Lexical Analyzer and Automata



Combine different NFAs



- How to combine different NFAs and convert them to only one NFA?!
  - Design each NFA separately.
    - Using Regular Expressions.
  - Glue them by Epsilon transitions "**Thompson's construction**".